

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**Веб-приложение на основе RAG для работы с документацией PostgreSQL
с учётом версии**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Студент: _____ / Жатиков Артур Русланович, 221–3210/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Гонтовой Сергей Викторович
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Тема ВКР	Веб-приложение на основе RAG для работы с документацией PostgreSQL с учётом версии
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Система обрабатывает пользовательский запрос по документации PostgreSQL с учётом выбранной версии СУБД. В кратком и подробном режимах формируются соответственно краткий и развёрнутый проверяемые ответы по официальному корпусу. В обучающем режиме формируется обучающий пошаговый материал с использованием официального корпуса и дополнительного корпуса как вспомогательного учебного слоя.

Основные функции	1. Система должна обеспечивать загрузку и хранение официальной документации PostgreSQL по нескольким версиям СУБД. 2. Система должна выполнять разбиение документации на текстовые фрагменты с сохранением метаданных: версия, раздел, заголовок, URL источника. 3. Система должна обеспечивать поиск релевантных фрагментов и повторное ранжирование по запросу пользователя с учётом выбранной версии PostgreSQL. 4. Система должна учитывать выбранную пользователем версию PostgreSQL при поиске и ранжировании фрагментов. 5. Система должна поддерживать три режима работы: • краткий режим — формирование краткого проверяемого ответа на основе официального корпуса; • подробный режим — формирование развёрнутого проверяемого ответа на основе официального корпуса; • обучающий режим — формирование обучающего пошагового материала с использованием официального и дополнительного корпусов. 6. Система должна отображать использованные источники с указанием разделов документации. 7. Пользовательский интерфейс должен обеспечивать ввод запроса, выбор версии PostgreSQL, выбор режима работы и просмотр сформированного результата.
Используемые технологии и платформы	Python 3, FastAPI, PostgreSQL, pgvector, модель векторных представлений, API языковой модели, HTML, CSS, JavaScript, Docker.

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Провести анализ предметной области и структуры документации PostgreSQL. 2. Исследовать влияние версии СУБД на корректность обработки запроса и формирования результата в кратком, подробном и обучающем режимах. 3. Сформулировать функциональные и нефункциональные требования к разрабатываемому веб-приложению. 4. Спроектировать RAG-архитектуру веб-приложения с учетом версии PostgreSQL и кратким, подробным и обучающим режимами ответа. 5. Разработать модель хранения документов, текстовых фрагментов и их метаданных. 6. Разработать алгоритмы индексации и обработки запроса с выбором режима, включая формирование текстовых ответов в кратком и подробном режимах и структурированного обучающего результата с учётом выбранной версии PostgreSQL. 7. Разработать пользовательский интерфейс системы. 8. Реализовать программный прототип системы. 9. Провести тестирование и оценку качества работы системы на наборе пользовательских запросов.
Состав технической документации	1. Пояснительная записка к выпускной квалификационной работе. 2. Описание предметной области. 3. Спецификация требований к системе. 4. Описание архитектуры системы. 5. Описание алгоритмов индексации, обработки запроса с выбором режима и формирования текстовых ответов в кратком и подробном режимах и структурированного обучающего результата. 6. Описание пользовательского интерфейса. 7. Руководство пользователя. 8. Описание результатов тестирования и оценки работы системы.
Состав графической части	1. UML-диаграмма последовательности взаимодействия компонентов системы 2. BPMN-диаграмма процесса обработки пользовательского запроса. 3. ER-диаграмма базы данных.

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«__»____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«__»____2026, _____ / Гонтовой Сергей Викторович /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«__»____2026, _____ / Жатиков Артур Русланович, 221–3210/
подпись *ФИО, группа*

АННОТАЦИЯ

Тема выпускной квалификационной работы — Веб-приложение на основе RAG для работы с документацией PostgreSQL с учетом версии.

Объект исследования — процесс предоставления пользователю справочной и обучающей информации по документации PostgreSQL с учетом версии СУБД.

Предмет исследования — решения веб-приложения, которое формирует ответы по локально индексируемому корпусу документации и возвращает использованные источники.

Цель работы — разработать веб-приложение, которое по запросу пользователя, версии PostgreSQL и режиму ответа извлекает релевантные фрагменты корпуса, проверяет подтверждения и формирует результат с источниками.

Рассмотрены особенности официальной документации PostgreSQL, проблема смешения сведений разных основных версий и подходы к поиску по техническим текстам. Спроектированы серверная часть, модель данных, подготовка корпуса, поиск фрагментов с учетом версии, повторное ранжирование и проверка подтверждений. В кратком и подробном режимах система использует официальный корпус, а в обучающем режиме подключает дополнительный корпус как вспомогательный источник.

Результат работы включает ответ, список источников и запись в истории запросов. Практическая значимость состоит в использовании приложения как прототипа сервиса для поиска и объяснения материалов документации PostgreSQL.

Ключевые слова: PostgreSQL, документация, версия СУБД, RAG, поиск фрагментов, повторное ранжирование, проверка источников, FastAPI, pgvector, веб-приложение.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	12
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ . .	16
1.1 Анализ предметной области	16
1.2 Особенности официальной документации PostgreSQL	17
1.3 Проблема учета версии PostgreSQL	18
1.4 Анализ существующих подходов поиска по технической документации	19
1.5 Сравнение существующих инструментов и конкурирующих решений	20
1.6 Риски генеративных ответов и необходимость подтверждения источниками	25
1.7 Анализ пользователей и сценариев использования	26
1.8 Постановка задачи	26
1.9 Функциональные требования	27
1.10 Нефункциональные требования	30
1.11 Вывод по главе	31
2 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	32
2.1 Общая архитектура системы	32
2.2 Архитектура серверной части	34
2.3 Сценарий обработки запроса	37
2.4 Диаграмма процесса обработки запроса	39
2.5 Модель данных и диаграмма сущность-связь	41
2.6 Алгоритм подготовки корпуса документации	45
2.7 Алгоритм разбиения документации на фрагменты	46
2.8 Механизм поиска фрагментов	46
2.9 Учет версии PostgreSQL при поиске	47
2.10 Повторное ранжирование найденных фрагментов	48

2.11	Проверка достаточности подтверждений	49
2.12	Обработка ошибок и безопасные сценарии	49
2.13	Вывод по главе	50
3	РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ	51
3.1	Выбор технологического стека	51
3.2	Структура программного проекта	51
3.3	Реализация модели базы данных	52
3.4	Реализация миграций и физической схемы PostgreSQL	54
3.5	Реализация маршрутов программного интерфейса	55
3.6	Реализация подготовки и индексации корпуса	56
3.7	Реализация обработки пользовательского запроса	57
3.8	Реализация поиска и учета версии	57
3.9	Реализация повторного ранжирования и проверки подтверждений	58
3.10	Реализация пользовательского интерфейса	59
3.11	Вывод по главе	60
4	ТЕСТИРОВАНИЕ И ОЦЕНКА РЕЗУЛЬТАТОВ	62
4.1	Цель и программа испытаний	62
4.2	Критерии проверки	64
4.3	Среда тестирования	64
4.4	Автоматические тесты	65
4.5	Сценарные проверки пользовательских функций	68
4.6	Проверка учета версии PostgreSQL	70
4.7	Проверка безопасного отказа при недостатке подтверждений	71
4.8	Проверка интерфейса и истории запросов	71
4.9	Результаты запуска тестов	72
4.10	Сопоставление результатов с требованиями	72
4.11	Ограничения разработанного решения	75
4.12	Вывод по главе	75
	ЗАКЛЮЧЕНИЕ	77

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	79
ПРИЛОЖЕНИЕ А	
Техническое задание	85
Общие сведения	85
Назначение разработки	85
Требования к функциональным характеристикам	86
Требования к данным и информационной совместимости	88
Требования к надежности и безопасности	88
Требования к программному и техническому обеспечению	89
Требования к программной документации	89
Стадии и этапы разработки	90
Порядок контроля и приемки	90
Ограничения разработки	91
Трассировка требований к реализации	91
ПРИЛОЖЕНИЕ Б	
Примеры запросов и ответов программного интерфейса	93
Пример запроса в кратком режиме (short)	93
Пример запроса в подробном режиме (detailed)	93
Пример запроса в обучающем режиме (tutorial)	93
Пример ответа в кратком режиме (short)	93
Пример ответа в обучающем режиме (tutorial)	94
ПРИЛОЖЕНИЕ В	
Руководство пользователя	96
Назначение пользовательского интерфейса	96
Получение ответа	96
Просмотр источников	97
Просмотр истории	97
Действия при ошибке	97

ПРИЛОЖЕНИЕ Г

Фрагменты листинга программного кода	99
Проверка версии официального источника	99
Выбор корпусов по режиму ответа	99

ПРИЛОЖЕНИЕ Д

Дополнительные результаты тестирования	101
Команда запуска тестов	101
Результат запуска	101

ВВЕДЕНИЕ

Официальная документация PostgreSQL является одним из основных рабочих источников для разработчиков, администраторов баз данных и инженеров эксплуатации. К ней обращаются при проектировании схем, проверке синтаксиса SQL-команд, настройке параметров сервера, диагностике репликации, анализе ограничений и сопровождении инсталляций в разных окружениях [1, 2, 3, 4]. Особенность этой документации состоит в том, что сведения публикуются по основным версиям PostgreSQL, а изменения поведения и набора возможностей фиксируются в политике версионирования и примечаниях к выпускам [5, 6]. Поэтому инженерный ответ по документации должен быть не только связным, но и проверяемым по источникам выбранной версии.

Актуальность темы. На практике пользователь редко формулирует запрос как точное название раздела документации. Запрос обычно описывает задачу: проверить параметр, понять поведение команды, найти способ диагностики или получить пошаговое объяснение. Обычный поиск по сайту документации или по локальному корпусу HTML-документации возвращает набор страниц, но не формирует итоговый ответ и не контролирует, из какой версии взяты сведения. Использование больших языковых моделей без привязки к источникам также недостаточно надежно для инженерной эксплуатации, поскольку модель может смешать сведения разных версий или сформулировать ответ, который не подтверждается документацией [7, 8, 9, 10, 11].

Подход генерации с дополненным извлечением (Retrieval-Augmented Generation, RAG) позволяет разделить задачу на поиск релевантных фрагментов и генерацию ответа по найденному контексту [7]. В работе этот подход применяется ограниченно и инженерно: корпус хранится локально, версия PostgreSQL передается как обязательный параметр, найденные источники возвращаются пользователю, а при недостатке подтверждений система не

должна формировать уверенную рекомендацию. Такая постановка соответствует общим принципам проектирования информационных систем и требованиям к качеству программного обеспечения [12, 13, 14].

Объект исследования – процесс предоставления пользователю справочной и обучающей информации по документации PostgreSQL с учетом выбранной версии.

Предмет исследования – архитектурные, алгоритмические и интерфейсные решения веб-приложения, которое выполняет поиск фрагментов документации PostgreSQL, формирует ответ с источниками и сохраняет историю запросов.

Цель работы – разработать веб-приложение, которое по запросу пользователя, выбранной версии PostgreSQL и режиму ответа извлекает релевантные фрагменты корпуса, проверяет достаточность подтверждений и формирует результат с источниками.

Для достижения цели поставлены следующие задачи:

1. проанализировать предметную область, структуру документации PostgreSQL и проблему учета версии;
2. сравнить существующие подходы поиска по технической документации и определить ограничения ответов без источников;
3. сформулировать функциональные и нефункциональные требования к веб-приложению;
4. спроектировать архитектуру серверной части, модель данных, сценарий обработки запроса и алгоритмы работы с корпусом;
5. реализовать хранение версий, документов, фрагментов, векторных представлений, истории запросов и источников в PostgreSQL;
6. реализовать маршруты API, подготовку корпуса, поиск фрагментов, контроль версии, повторное ранжирование и проверку достаточности подтверждений;

7. реализовать пользовательский интерфейс с вводом вопроса, выбором версии, выбором режима ответа, отображением результата, источников и истории;

8. провести автоматическое и сценарное тестирование реализованных функций без заявления неподтвержденных количественных метрик качества ранжирования.

Работа состоит из четырех глав. В первой главе приведен анализ предметной области, документации PostgreSQL, существующих подходов поиска, рисков генеративных ответов, требований и постановки задачи. Во второй главе описаны проектные решения: архитектура веб-приложения, сценарий обработки запроса, диаграммы, модель данных и алгоритмы подготовки корпуса, поиска фрагментов, учета версии, повторного ранжирования и проверки подтверждений. В третьей главе описана реализация серверной части, базы данных, маршрутов API, индексации, обработки запроса и пользовательского интерфейса. В четвертой главе приведены программа испытаний, автоматические тесты, сценарные проверки, результаты запуска тестов, сопоставление с требованиями и ограничения разработанного решения.

Методы исследования. В работе использованы анализ предметной области, сравнительный анализ существующих подходов, структурно-функциональное моделирование, проектирование модели данных, проектирование программной архитектуры и функциональное тестирование. Для обоснования поисковой части учтены исследования в области информационного поиска, векторных представлений текста и вопросно-ответных систем [15, 16, 17, 18]. Для проектирования данных использованы классические работы по реляционной модели и базам данных [19, 20, 21, 22, 23, 24]. Для архитектуры и сопровождения программной системы учтены подходы к разделению ответственности, эволюции кода и архитектурным стилям [25, 26, 27, 28, 29, 30, 31]. Для проверки использованы общие принципы функционального тестирования [32, 33, 34].

Практическая значимость. Разработанная система может использоваться как прототип сервиса поиска и объяснения материалов по документации PostgreSQL. Для разработчика приложений она помогает быстрее найти проверяемый ответ по SQL-командам и системным представлениям. Для администратора баз данных система полезна при проверке параметров, репликации, обслуживания и ограничений конкретной версии. Для инженера эксплуатации интерфейс с источниками и историей запросов упрощает повторную проверку рекомендаций при сопровождении PostgreSQL в локальной или контейнерной инфраструктуре [35, 36, 37].

В практической части реализована программная система на Python 3.12 с использованием FastAPI, SQLAlchemy, Alembic, PostgreSQL, pgvector, Pydantic, BeautifulSoup, httpx, Groq API и pytest [38, 39, 40, 41, 42, 43, 44, 45, 46]. Фактическая реализация поддерживает версии PostgreSQL 16, 17 и 18, краткий (short), подробный (detailed) и обучающий (tutorial) режимы ответа, локальную модель построения векторных представлений BAAI/bge-m3 размерности 1024 и хранение истории пользовательских запросов.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Анализ предметной области

Предметная область работы связана с поиском и объяснением сведений из технической документации PostgreSQL. PostgreSQL используется как промышленная система управления базами данных, поэтому документация охватывает широкий набор тем: SQL-команды, типы данных, администрирование, параметры сервера, репликацию, расширения, утилиты и примечания к выпускам [1, 21, 22, 23]. Пользовательские вопросы в такой области редко совпадают с точным названием страницы документации. Обычно вопрос описывает прикладную задачу: найти активные запросы, проверить параметр репликации, объяснить план выполнения, отличить VACUUM от ANALYZE, подобрать источник для дальнейшей ручной проверки.

Техническая документация отличается от справочного текста общего назначения. Для нее важны точность, воспроизводимость, связь с конкретным источником и учет контекста версии. Ошибка в ответе по настройке PostgreSQL может привести не только к неверному пониманию механизма, но и к некорректной конфигурации рабочей базы данных. Поэтому система, формирующая ответ по документации, должна сохранять трассируемость результата: пользователь должен видеть, какие документы и разделы были использованы.

Поиск по документации можно рассматривать как частный случай информационного поиска [15]. Классические лексические методы хорошо работают, когда запрос содержит точные термины, но хуже обрабатывают переформулировки. Семантический поиск по векторным представлениям лучше переносит разные формулировки, однако сам по себе не гарантирует версию корректность и не формирует связный ответ. Поэтому в работе рассматривается комбинированное решение: локальный корпус документации,

поиск фрагментов, повторное ранжирование, проверка достаточности подтверждений и генерация результата с возвращением источников.

1.2 Особенности официальной документации PostgreSQL

Официальная документация PostgreSQL обладает устойчивой структурой и публикуется по версиям [1, 2, 3, 4]. Для каждой основной версии существует отдельная ветка URL вида `/docs/<version>/`. Такая организация удобна для пользователя, но требует аккуратной обработки в программной системе: ссылка на страницу текущей версии или на другую ветку документации не должна незаметно подменять выбранную пользователем версию.

С точки зрения подготовки корпуса документация имеет несколько полезных свойств. Во-первых, страницы содержат заголовки разных уровней, которые можно преобразовать в путь раздела. Во-вторых, разделы включают обычные абзацы, списки, таблицы и фрагменты кода. В-третьих, URL документа часто содержит семантический якорь, например название SQL-команды или раздела конфигурации. Эти свойства позволяют сохранять в системе не только текст фрагмента, но и метаданные: заголовок документа, путь раздела, URL источника, тип корпуса и версию.

Официальный корпус в разработанном приложении является основным источником фактов. Дополнительный учебный корпус используется только как вспомогательный слой в обучающем режиме и не должен заменять официальные источники. Такое разграничение соответствует инженерному характеру задачи: объяснение может быть адаптировано для начинающего пользователя, но фактическая часть должна проверяться по документации PostgreSQL.

1.3 Проблема учета версии PostgreSQL

Политика версионирования PostgreSQL определяет основную версию как первую часть номера версии, начиная с PostgreSQL 10 [5]. Между основными версиями могут изменяться возможности, ограничения, параметры конфигурации, состав утилит и рекомендации по эксплуатации. Поэтому запрос вида «как настроить логическую репликацию» не является полностью определенным без указания версии PostgreSQL.

В практической реализации поддерживаются версии PostgreSQL 16, 17 и 18. Они заданы в конфигурации через `SUPPORTED_PG_VERSIONS=16,17,18`, заносятся в таблицу `versions` и возвращаются маршрутом `/api/versions`. Версия участвует в валидации входного запроса, выборе документов и дополнительной проверке URL официальных источников. Если пользователь указывает неподдерживаемую версию, запрос отклоняется на уровне схемы данных или при разрешении версии.

Основной риск при отсутствии контроля версии показан в таблице 1. В таблице намеренно рассматриваются не частные ошибки конкретного фрагмента кода, а классы проблем, которые должна предотвращать система.

Таблица 1 – Риски отсутствия учета версии PostgreSQL

Риск	Проявление	Требуемая реакция системы
Смещение веток документации	В контекст одного ответа попадают страницы <code>/docs/16/</code> , <code>/docs/17/</code> и <code>/docs/18/</code> .	Фильтрация документов по выбранной версии и дополнительная проверка URL официальных источников.
Подмена текущей версией	Ссылка <code>/docs/current/</code> попадает в ответ для фиксированной версии.	Отсечение официальных источников, не содержащих ожидаемый фрагмент <code>/docs/<version>/</code> .
Слабый контекст	Найденные фрагменты имеют общие слова, но не подтверждают технический вывод.	Проверка достаточности подтверждений перед генерацией уверенного ответа.

Продолжение таблицы 1

Риск	Проявление	Требуемая реакция системы
Неверное учебное объяснение	Дополнительный материал вытесняет официальный источник в обучающем режиме.	Приоритет официального корпуса и использование дополнительного корпуса только как вспомогательного.

1.4 Анализ существующих подходов поиска по технической документации

Поиск по технической документации может строиться разными способами. Обычный поиск по сайту удобен для навигации, но требует ручной интерпретации страниц. Полнотекстовый поиск использует токенизацию, нормализацию и ранжирование документов; в PostgreSQL такие возможности представлены встроенными механизмами полнотекстового поиска [47]. Семантический поиск использует векторные представления и расстояние между векторами, что позволяет находить близкие по смыслу фрагменты [8, 9]. Комбинированные методы объединяют лексический и семантический сигнал, а повторное ранжирование корректирует порядок кандидатов [16, 10].

Для задач вопросно-ответного взаимодействия применяется подход RAG: система сначала извлекает контекст из внешнего корпуса, затем передает его генеративной модели [7]. В работе этот подход не трактуется как универсальное средство автоматической истины. Он используется как инженерный способ связать генерацию ответа с локально контролируемыми источниками, ограничить контекст выбранной версией и вернуть пользователю список источников.

Сравнение классов методов приведено в таблице 2. Из таблицы следует, что ни один из подходов по отдельности не закрывает все требования: связный ответ, источники, локальный контроль корпуса и снижение риска смешения версий достигаются только при сочетании поиска фрагментов, контроля версии и проверки подтверждений.

Таблица 2 – Сравнение подходов к поиску по технической документации

Подход	Принцип работы	Преимущества	Ограничения
Поиск по ключевым словам	Поиск точных совпадений терминов в тексте документа.	Простота реализации и понятность результата.	Низкая устойчивость к перефразировкам и синонимам.
Полнотекстовый поиск	Индексация текста с нормализацией словоформ и ранжированием документов.	Хорошо подходит для терминологических запросов и локального корпуса.	Не формирует итоговый ответ и не проверяет достаточность источников.
Семантический поиск	Сравнение векторного представления запроса с векторными представлениями фрагментов.	Лучше обрабатывает близкие по смыслу формулировки.	Может выбрать тематически близкий, но версионно неверный фрагмент без контроля версии.
Комбинированный поиск	Объединение семантических и лексических кандидатов.	Повышает устойчивость при технических терминах и свободной формулировке.	Требует логики объединения, фильтрации и повторного ранжирования.
RAG с источниками	Генерация ответа только после извлечения контекста из корпуса.	Формирует связный результат и позволяет вернуть источники.	Качество зависит от корпуса, поиска фрагментов и правил проверки контекста.

1.5 Сравнение существующих инструментов и конкурирующих решений

Для обоснования направления решения сравнивались не конкретные коммерческие продукты, а классы инструментов, которыми пользователь обычно пользуется при работе с документацией PostgreSQL. Такой подход корректен для инженерной ВКР, поскольку задача состоит не в маркетинговом сравнении, а в выявлении функционального разрыва между доступными способами поиска и требуемым поведением системы.

Критерии сравнения выбраны из требований предметной области: учет версии, наличие источников, формирование связного ответа, риск смешения версий, локальный контроль корпуса и пригодность для инженерного исполь-

зования. Результат сравнения приведен в таблице 3. В последней строке указано направление, реализованное в практической части проекта.

Обычный поиск по сайту документации PostgreSQL остается полезным при ручной проверке, но он не решает задачу сопровождения ответа источниками в одном пользовательском сценарии. Пользователь получает набор страниц и сам выбирает, какие фрагменты относятся к его вопросу. Для опытного специалиста это приемлемо, однако начинающий пользователь может пропустить важное ограничение версии или принять общий раздел за ответ на частный вопрос. Кроме того, при переходах между страницами легко смешать материалы разных веток документации, особенно если часть ссылок ведет на текущую версию.

Полнотекстовый поиск в локальном корпусе лучше подходит для контролируемой среды, поскольку документы можно заранее разделить по версиям и хранить вместе с метаданными. Его ограничение состоит в том, что терминологически близкий запрос не всегда содержит те же слова, что и документация. Например, пользователь может спрашивать о «зависших запросах», а документация описывает системное представление `pg_stat_activity` и состояние запроса другими терминами. В такой ситуации только лексического совпадения недостаточно: требуется поиск по смысловой близости, а затем дополнительная проверка найденных фрагментов.

Выбор RAG-подхода с учетом версии связан именно с этим разрывом между ручным поиском и неконтролируемой генерацией. В разработанном решении генеративная модель не заменяет документацию и не выступает самостоятельным источником знаний. Она получает уже отобранный контекст, ограниченный выбранной основной версией PostgreSQL, а результат возвращается вместе со списком использованных источников. Поэтому ключевым становится не сам факт применения генерации, а порядок обработки запроса: сначала валидация версии и поиск фрагментов, затем повторное ранжирование, проверка подтверждений и только после этого формирование ответа.

Такой подход также объясняет разделение официального и дополнительного корпусов. Официальная документация используется как основной источник фактов для краткого и подробного режимов. Дополнительный корпус допустим только в обучающем режиме, где требуется более плавное объяснение и последовательность шагов. При этом дополнительный материал не должен вытеснять официальный источник из ответа: если официального подтверждения недостаточно, система должна отказаться от уверенной формулировки. Это ограничение снижает риск того, что удобное учебное объяснение будет принято за проверенный факт без связи с документацией PostgreSQL.

Таблица 3 – Сравнение инструментов и подходов для поиска по документации PostgreSQL

Подход	Учет версии	Источники	Связный ответ	Риск смещения версий	Локальный контроль корпуса	Инженерная пригодность
Поиск по сайту документации PostgreSQL	Возможен вручную через выбор ветки документации.	Есть ссылки на страницы.	Не формируется, пользователь анализирует страницы самостоятельно.	Средний: пользователь может перейти в другую ветку или на текущую версию.	Нет, используется внешний сайт.	Высокая для ручной проверки, средняя для быстрого ответа.
Полнотекстовый поиск в локальном корпусе	Возможен при отдельном индексе по версии.	Можно вернуть путь к документу.	Не формируется без дополнительной логики.	Низкий при правильной схеме индексации.	Да.	Высокая как базовый компонент, недостаточная как самостоятельный помощник.
Общие поисковые системы	Не гарантируется.	Есть ссылки, но могут вести на разные версии и сторонние материалы.	Частично через сниппеты, без инженерной проверки.	Высокий.	Нет.	Низкая для ответов, средняя для первичного поиска.
Поиск в среде разработки или локальном корпусе HTML-документации	Зависит от того, какой корпус загружен локально.	Есть путь к файлу или URL.	Не формируется.	Низкий при одном корпусе, высокий при смешанной папке.	Да.	Высокая для опытного пользователя, ниже для начинающего.
Вопросно-ответная система без привязки к источникам	Обычно не контролируется пользователем.	Источники могут отсутствовать или быть неполными.	Формируется.	Высокий.	Нет.	Ограниченная: требуется ручная перепроверка.

Продолжение таблицы 3

Подход	Учет версии	Источники	Связный ответ	Риск смещения версий	Локальный контроль корпуса	Инженерная пригодность
RAG с учетом версии и источниками	Версия является обязательным параметром запроса.	Возвращаются документы, разделы, URL и позиция источника.	Формируется в выбранном режиме ответа.	Снижается за счет фильтрации и проверки источников.	Да, корпус индексируется локально.	Подходит для реализованных сценариев при наличии полного корпуса.

1.6 Риски генеративных ответов и необходимость подтверждения источниками

Генеративная модель может построить грамматически убедительный ответ даже тогда, когда контекст недостаточен. В инженерной задаче это является существенным риском: пользователь может принять неподтвержденную формулировку за факт. Поэтому разработанная система не должна использовать генерацию как самостоятельный источник знаний. Генерация допустима только после извлечения фрагментов из корпуса и проверки, что среди них есть официальные источники выбранной версии.

В приложении эта идея выражается в трех требованиях. Во-первых, ответ должен сопровождаться списком источников. Во-вторых, в кратком (short) и подробном (detailed) режимах контекст строится только на официальном корпусе. В-третьих, в обучающем режиме (tutorial) дополнительный корпус может улучшать учебную подачу, но официальный корпус должен оставаться основой фактического вывода. Подобные ограничения соответствуют общим принципам проектирования надежных информационных систем: ограничения должны быть выражены не только в пользовательской инструкции, но и в программной логике [25, 26, 28].

Отдельно необходимо учитывать отсутствие количественной оценки качества ранжирования в этой ВКР. Поэтому нельзя заявлять полноту поиска, абсолютную точность или численные метрики вида Recall@K, MRR, nDCG. Проверка качества выполняется функционально и сценарно: подтверждается, что реализованные правила выбора версии, режима, источников и безопасного отказа работают согласно требованиям.

1.7 Анализ пользователей и сценариев использования

Пользователями приложения являются разработчики, администраторы баз данных, инженеры эксплуатации и начинающие специалисты. Для разработчика важны быстрые ответы по SQL-командам, системным представлениям и типичным диагностическим запросам. Для администратора баз данных важны параметры сервера, обслуживание, репликация и ограничения версии. Для инженера эксплуатации важны воспроизводимые инструкции, контейнерное окружение и история выполненных запросов. Для начинающего пользователя важна пошаговая подача материала и пояснение предварительных условий.

Основные пользовательские сценарии приведены в таблице 4. Они служат основой для требований и последующей программы испытаний.

Таблица 4 – Пользовательские сценарии приложения

Пользователь	Типовой запрос	Ожидаемое поведение системы
Разработчик	«Напиши SQL-запрос для активных запросов дольше пяти минут».	Краткий или подробный ответ с проверяемыми источниками и, если уместно, блоком SQL.
Администратор баз данных	«Какие параметры проверить для логической репликации в PostgreSQL 16?»	Поиск официальных источников выбранной версии и отказ от смешения с другими версиями.
Инженер эксплуатации	«Как проверить блокировки и найти блокирующий запрос?»	Подробный ответ с источниками и возможностью найти запись в истории.
Начинающий специалист	«Объясни EXPLAIN ANALYZE простыми словами».	Обучающий результат с кратким объяснением, предварительными условиями, шагами и замечаниями.

1.8 Постановка задачи

Необходимо разработать веб-приложение для работы с документацией PostgreSQL, которое принимает вопрос пользователя, выбранную основную

версию PostgreSQL и режим ответа. Система должна подготовить локальный корпус, сохранить документы и фрагменты в базе данных, вычислить векторные представления, выполнить поиск фрагментов, применить контроль версии, выполнить повторное ранжирование, проверить достаточность подтверждений и сформировать ответ с источниками.

Практическая часть ограничивается реализованными функциями. В ней нет пользовательского переключателя корпуса: выбор корпуса определяется режимом ответа. Для краткого (short) и подробного (detailed) режимов используется официальный корпус. Для обучающего режима (tutorial) используется официальный корпус и дополнительный учебный корпус, причем дополнительный корпус не должен вытеснять официальный. Количественная оценка качества ранжирования не выполнялась, поэтому результат оценивается функционально и сценарно.

Задача разработки включает серверную часть, модель данных, маршруты API, офлайн-подготовку корпуса, онлайн-обработку запроса, пользовательский интерфейс и автоматические тесты. Набор поддерживаемых версий фиксируется конфигурацией проекта, а поддержка новых версий предполагает наличие соответствующего корпуса и переиндексацию.

1.9 Функциональные требования

Функциональные требования сформулированы на основе постановки задачи и сверены с фактической реализацией. Требования приведены в таблице 5. В таблице не перечисляются все внутренние файлы проекта, поскольку требования должны описывать поведение системы, а не структуру репозитория.

Таблица 5 – Функциональные требования к веб-приложению

ID	Требование	Описание	Проверка
FR-01	Поддержка версий PostgreSQL	Система должна хранить и возвращать поддерживаемые основные версии PostgreSQL 16, 17 и 18.	Маршрут /api/versions, тесты API.
FR-02	Подготовка официального корпуса	Система должна загружать локальные HTML-страницы официальной документации по версии.	Тесты загрузчика официально-го корпуса.
FR-03	Разбиение документации на фрагменты	Система должна выделять текстовые фрагменты с путем раздела, типом содержимого, числом токенов и педагогической ролью.	Тесты парсера и разбиения.
FR-04	Вычисление и хранение векторных представлений	Для каждого фрагмента должен сохраняться вектор размерности 1024, рассчитанный моделью BAAI/bge-m3.	Тесты правил построения векторных представлений и миграции.
FR-05	Поиск фрагментов по версии	Поиск должен учитывать выбранную версию и допустимые типы корпуса.	Тесты контроля версии и интеграционные тесты.
FR-06	Повторное ранжирование	Найденные кандидаты должны упорядочиваться с учетом семантического, лексического и терминологического сигналов.	Тесты повторного ранжирования.
FR-07	Краткий режим (short)	Система должна формировать краткий ответ по официальному корпусу и возвращать источники.	Тесты маршрута /api/ask.
FR-08	Подробный режим (detailed)	Система должна формировать подробный ответ по официальному корпусу и возвращать источники.	Тесты маршрута /api/ask.
FR-09	Обучающий режим (tutorial)	Система должна формировать структуру short_explanation, prerequisites, steps, notes с использованием официального и дополнительного корпуса.	Тесты API и генерации.
FR-10	Проверка достаточности подтверждений	При слабом официальном контексте система должна возвращать безопасный отказ вместо уверенного ответа.	Тесты проверки подтверждений.
FR-11	История запросов	Система должна сохранять вопрос, версию, режим, статус, задержку, ответ и источники.	Маршрут /api/history, тесты истории.
FR-12	Служебная диагностика	Система должна предоставлять маршруты проверки состояния приложения и модуля векторных представлений.	Маршруты /api/health, /api/health/embeddings.

Продолжение таблицы 5

ID	Требование	Описание	Проверка
FR-13	Административная переиндексация	Система должна предоставлять административный маршрут запуска переиндексации корпуса с проверкой ключа при заданном ADMIN_API_KEY.	Маршрут /api/admin/reindex, тесты сценария.
FR-14	Пользовательский интерфейс	Интерфейс должен обеспечивать ввод вопроса, выбор версии, выбор режима, просмотр ответа, источников и истории.	Тесты шаблонов и сценарная проверка.

1.10 Нефункциональные требования

Нефункциональные требования приведены в таблице 6. Они определяют ограничения качества, воспроизводимости и проверяемости, но не вводят неподтвержденных метрик точности.

Таблица 6 – Нефункциональные требования к веб-приложению

ID	Требование	Описание
NFR-01	Трассируемость ответа	Ответ должен сопровождаться источниками с названием, URL, типом корпуса, разделом, позицией и оценкой релевантности.
NFR-02	Версионная изоляция	Официальные источники другой версии или /docs/current/ не должны попадать в ответ выбранной версии.
NFR-03	Воспроизводимость развертывания	Серверная часть и PostgreSQL должны запускаться через Docker Compose с явной конфигурацией окружения [35].
NFR-04	Контролируемость генерации	Генерация должна выполняться только по переданному контексту, а источники возвращаются отдельно от текста ответа.
NFR-05	Тестируемость	Критичные правила должны проверяться автоматическими тестами, запускаемыми командой pytest [46, 32, 33].
NFR-06	Безопасность конфигурации	Секреты генеративной модели и административный ключ должны передаваться через переменные окружения, а не храниться в коде.
NFR-07	Расширяемость корпуса	Добавление новой версии должно выполняться через добавление корпуса и переиндексацию без изменения маршрута пользовательского запроса.
NFR-08	Ограниченность оценки	Оценка результатов должна быть функциональной и сценарной; количественные метрики ранжирования не заявляются без отдельного набора эталонных запросов.

Нормативная база отчета. Структура пояснительной записки, оформление списка источников, библиографических ссылок, технического задания и критериев качества программного обеспечения соотнесены с требованиями стандартов СИБИБД, ЕСПД и стандартов жизненного цикла автоматизированных систем [48, 49, 50, 51, 52, 53, 14].

1.11 Вывод по главе

В первой главе проанализирована предметная область поиска по документации PostgreSQL, выявлена проблема учета основной версии и рассмотрены существующие подходы к поиску по технической документации. Сравнение показало, что обычный поиск, полнотекстовый поиск, локальный поиск и вопросно-ответные системы без источников не закрывают одновременно требования версионной корректности, связного ответа, локального контроля корпуса и трассируемости источников. На этой основе сформулирована постановка задачи и определены функциональные и нефункциональные требования к веб-приложению.

2 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Общая архитектура системы

Архитектура веб-приложения показана на рисунке 1. На схеме выделены две основные части: контур подготовки корпуса и серверная часть. Такое разделение используется потому, что подготовка документации и обработка пользовательского запроса выполняются в разное время. Ресурсоемкие операции очистки, разбиения и векторизации вынесены из пользовательского сценария, а серверная часть работает с уже подготовленным индексом [25, 26, 54].

Контур подготовки корпуса отвечает за предварительную работу с материалами. В него поступает документация PostgreSQL по версиям 16, 17 и 18, а также дополнительные учебные материалы. Эти данные очищаются от лишних элементов, разбиваются на отдельные текстовые фрагменты и переводятся в векторные представления с помощью модели BAAI/bge-m3. После этого документы, фрагменты, векторы и связанные с ними сведения сохраняются в PostgreSQL с расширением pgvector.

Серверная часть работает с запросом пользователя. Пользователь вводит вопрос в пользовательском интерфейсе, выбирает версию PostgreSQL и режим ответа. Затем запрос передается в FastAPI, после чего сервис обработки запроса последовательно выполняет анализ вопроса, построение векторного представления, поиск подходящих фрагментов, ранжирование и проверку найденных источников.

Центральный компонент обработки находится в RAG-контуре. Он обращается к базе данных, получает фрагменты документации нужной версии и проверяет, подходят ли они для ответа. Двухнаправленная связь между RAG-контуром и базой данных показывает, что система работает с сохраненными версиями, документами, фрагментами, векторами и источниками. После

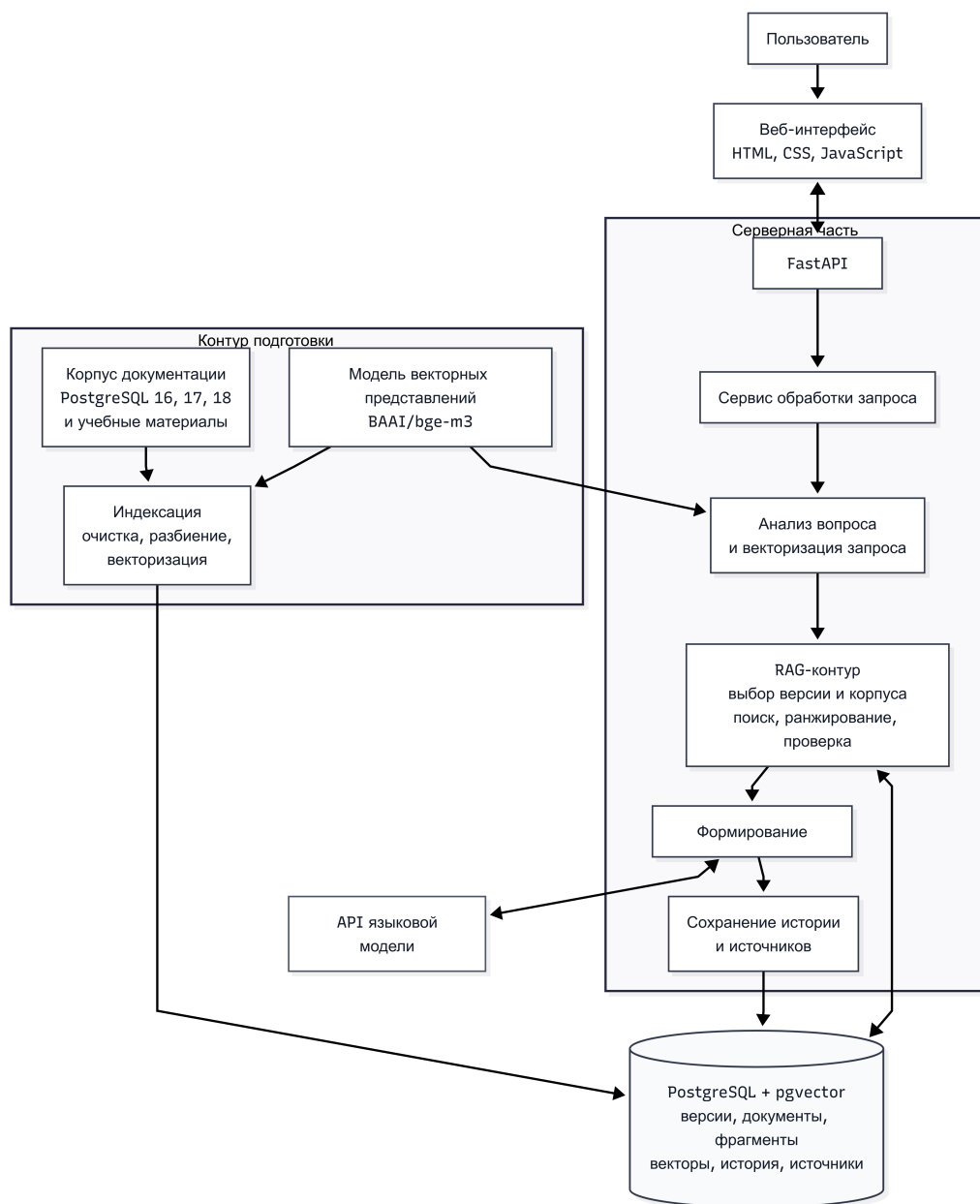


Рисунок 1 – Архитектура веб-приложения для работы с документацией PostgreSQL с учетом версии

проверки найденного контекста компонент формирования ответа обращается к внешней языковой модели Groq: приложение передает подготовленный запрос и найденные фрагменты, а получает сформированный текст ответа. При этом языковая модель не используется как самостоятельный источник знаний. Ответ строится на основе фрагментов, найденных в локальной базе документации.

После формирования результата система сохраняет историю запроса и использованные источники. Благодаря этому пользователь получает не только ответ, но и сведения о материалах документации, на которые он опирается. Такая архитектура решает три основные задачи приложения: учитывать выбранную версию PostgreSQL, возвращать проверяемые источники и снижать риск неподтвержденных ответов.

В реализации этим компонентам соответствуют FastAPI-приложение, PostgreSQL с расширением pgvector, ORM-модели SQLAlchemy, миграции Alembic и схемы данных Pydantic [38, 39, 40, 41, 42]. Генерация итогового текста выполняется через Groq API, а векторные представления формируются локальной моделью BAAI/bge-m3.

2.2 Архитектура серверной части

Серверная часть проектируется как набор слоев с явным разделением ответственности. Маршруты API принимают HTTP-запросы и передают данные в сервисы. Pydantic-схемы определяют контракт входных и выходных структур. Сервисный слой реализует правила обработки запроса. Репозитории инкапсулируют SQL-запросы и сохранение истории. Слой базы данных содержит ORM-модели и подключение к PostgreSQL.

Роли основных компонентов приведены в таблице 7. Компоненты выбраны по фактической структуре проекта, но таблица описывает их на архитектурном уровне, без избыточного перечисления внутренних файлов.

Таблица 7 – Компоненты серверной части

Компонент	Назначение	Связь с обработкой запроса
Маршруты API	Прием запросов, вызов сервисов, возврат ответа клиенту.	Запускают сценарии /api/ask, /api/versions, /api/history, диагностики и переиндексации.
Схемы данных	Описание допустимых полей, типов и ограничений.	Фиксируют краткий (short), подробный (detailed) и обучающий (tutorial) режимы и структуру источников.
Сервисы	Бизнес-логика обработки вопроса, подготовки корпуса и истории.	Выполняют анализ вопроса, поиск фрагментов, повторное ранжирование, проверку подтверждений и генерацию.
Репозитории	Доступ к таблицам и SQL-выборкам.	Получают кандидаты поиска, сохраняют запросы и связанные источники.
База данных	Хранение версий, документов, фрагментов, векторных представлений и истории.	Обеспечивает версиюнную фильтрацию, векторный поиск и трассируемость результата.
Пользовательский интерфейс	HTML-страницы, стили и сценарии браузера.	Передает вопрос, версию и режим ответа, отображает ответ, источники и историю.

Проектные маршруты API приведены в таблице 8. Кроме маршрутов с префиксом /api, приложение предоставляет HTML-страницы / и /history, а также дублирующие маршруты диагностики /health и /health/embeddings без префикса API.

Таблица 8 – Проектный состав маршрутов API

Маршрут API	Метод	Назначение	Основные входные данные	Результат
/api/ask	POST	Основной пользовательский запрос к приложению.	question, pg_version, answer_mode.	Ответ или обучающая структура; список источников.
/api/versions	GET	Получение поддерживаемых версий PostgreSQL.	Нет обязательных параметров.	Список версий с docs_base_url и признаком поддержки.
/api/history	GET	Просмотр истории запросов.	Параметр limit.	Записи истории с вопросом, версией, режимом, статусом, задержкой и источниками.
/api/health	GET	Базовая диагностика состояния API.	Нет обязательных параметров.	status, timestamp.
/api/health/embeddings	GET	Диагностика модуля векторных представлений.	Нет обязательных параметров.	Поставщик, модель, размерность, размер пакета, окружение и статус.
/api/admin/reindex	POST	Запуск переиндексации корпуса.	versions, include_official, include_supplementary, max_pages, заголовок X-Admin-Key при заданном ADMIN_API_KEY.	Статус запуска и краткая статистика по версиям.

2.3 Сценарий обработки запроса

Сценарий обработки запроса начинается в пользовательском интерфейсе. Пользователь вводит вопрос, выбирает версию PostgreSQL и один из трех режимов ответа. Браузер отправляет на `/api/ask` только поля `question`, `pg_version`, `answer_mode`; отдельного флага для подключения дополнительного корпуса нет.

После валидации серверная часть проверяет, поддерживается ли версия. Затем выполняется анализ вопроса: нормализация, определение намерения, выделение технических терминов и формирование расширенного набора терминов для лексической ветки поиска.

Далее система выбирает типы корпусов. Для краткого (`short`) и подробного (`detailed`) режимов выбирается официальный корпус (`official`); для обучающего режима (`tutorial`) выбираются официальный и дополнительный корпус (`official`, `supplementary`). После этого вопрос преобразуется в векторное представление, выполняется поиск фрагментов в PostgreSQL, применяется контроль версии, результаты повторно ранжируются, а затем выполняется проверка достаточности подтверждений. Только после этого вызывается генерация ответа или формируется безопасный отказ.

Диаграмма последовательности на рисунке 2 показывает взаимодействие пользователя, интерфейса, маршрута API, сервисов, базы данных и генеративного адаптера.

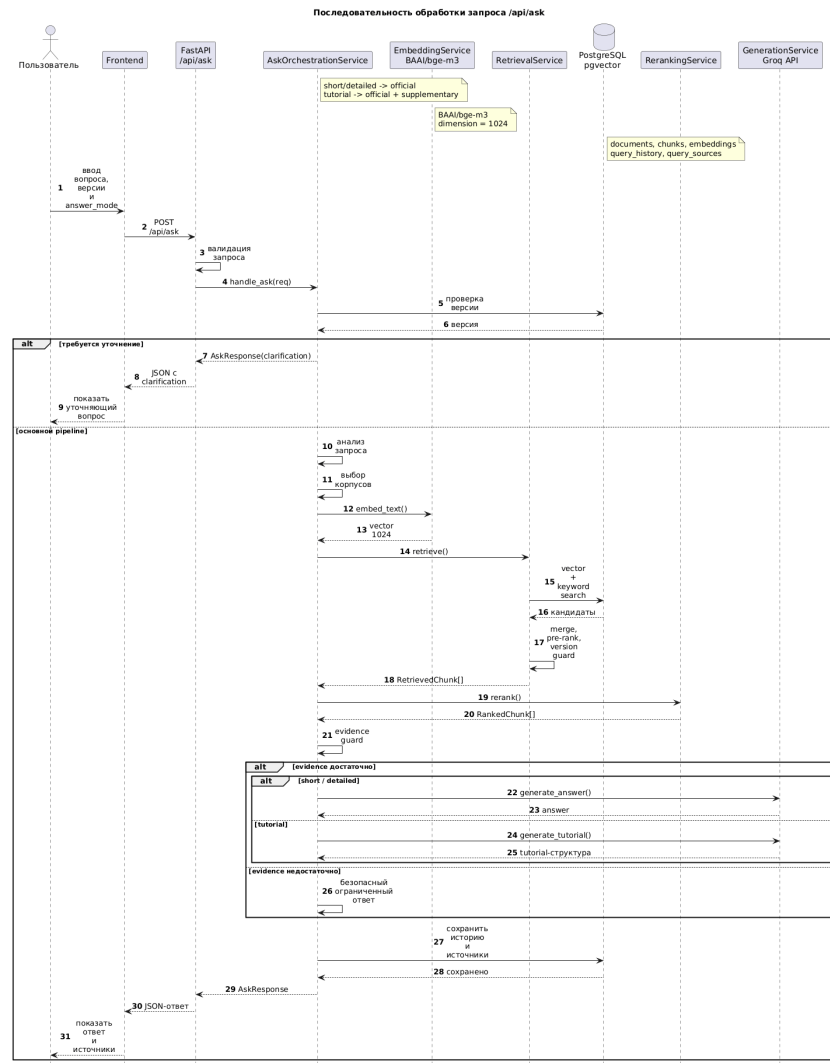


Рисунок 2 – UML-диаграмма последовательности взаимодействия компонентов системы

2.4 Диаграмма процесса обработки запроса

Процесс обработки пользовательского запроса представлен на рисунке 3. Диаграмма показывает ветвление по режиму ответа, проверку источников и разные варианты завершения: успешный ответ, обучающий результат, безопасный отказ или ошибка валидации.

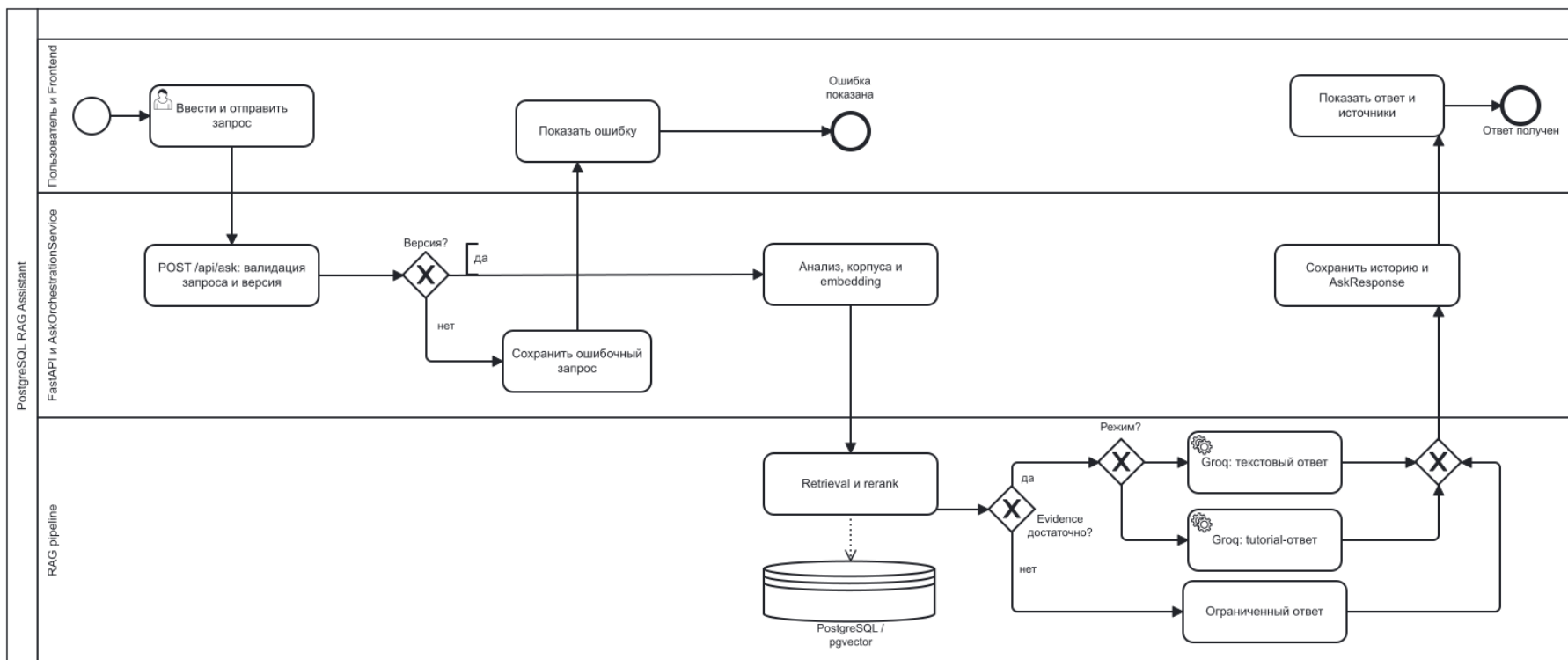


Рисунок 3 – BPMN-диаграмма процесса обработки пользовательского запроса

2.5 Модель данных и диаграмма сущность-связь

Модель данных проектируется вокруг трассируемости результата. Ответ должен быть связан не только с текстом вопроса, но и с версией PostgreSQL, режимом ответа и источниками, которые были использованы при генерации. Поэтому база данных содержит таблицы версий, документов, фрагментов, векторных представлений, истории запросов и источников запроса.

Диаграмма сущность-связь показана на рисунке 4. Она отражает фактическую схему, реализованную ORM-моделями и миграциями Alembic.



Рисунок 4 – ER-диаграмма базы данных

Состав основных таблиц приведен в таблице 9. Все первичные ключи реализованы типом UUID, структура обучающего результата хранится как JSONB, а векторные представления хранятся в отдельной таблице с типом vector(1024) [55, 56, 42].

Таблица 9 – Проектная модель данных

Таблица	Ключевые поля	Назначение	Связи
versions	id, major_version, docs_base_url, is_supported, loaded_at.	Хранение поддерживаемых основных версий PostgreSQL.	Связана с documents и query_history.
documents	id, version_id, title, source_url, checksum, corpus_type, audience_level.	Хранение страниц официального и дополнительного корпуса.	Связана с versions и chunks.
chunks	id, document_id, chunk_index, section_path, chunk_text, token_count, content_type, pedagogical_role.	Хранение текстовых фрагментов документации.	Связана с documents, embeddings, query_sources.
embeddings	id, chunk_id, embedding_model, embedding_dimension, embedding, indexed_at.	Хранение векторных представлений фрагментов.	Один фрагмент имеет один вектор.
query_history	id, selected_version_id, user_question, mode, answer_text, tutorial_json, status, latency_ms.	Хранение истории пользовательских запросов.	Связана с versions и query_sources.
query_sources	id, query_id, chunk_id, rank_position, similarity_score, source_role, source_url, corpus_type.	Трассировка ответа до использованных источников.	Связана с query_history и chunks.

2.6 Алгоритм подготовки корпуса документации

Подготовка корпуса выполняется как отдельный процесс переиндексации. Он может запускаться из командной строки или через административный маршрут `/api/admin/reindex`. Входными параметрами являются список версий, признаки включения официального и дополнительного корпуса, а также необязательное ограничение числа страниц.

Алгоритм подготовки корпуса приведен в листинге 2.1. Он отражает проектную логику без привязки к конкретным строкам кода. Листинг 2.1 – Алгоритм подготовки корпуса документации

```
1 для каждой выбранной версии PostgreSQL:
2     получить или создать запись версии в базе данных
3     если выбран официальный корпус:
4         загрузить HTML-страницы из corpus/postgres/html/<version>
5         очистить HTML и выделить структурные блоки
6         разбить текст на фрагменты
7         вычислить векторные представления фрагментов
8         заменить ранее сохраненный официальный корпус этой версии
9     если выбран дополнительный корпус:
10        загрузить markdown-документы из corpus/tutorial
11        проверить метаданные и пригодность к индексации
12        разбить текст на фрагменты
13        вычислить векторные представления фрагментов
14        заменить ранее сохраненный дополнительный корпус этой версии
```

Особенность реализованного процесса состоит в том, что тяжелая подготовка и вычисление векторных представлений выполняются до удаления старых данных текущего корпуса. После подготовки данные сохраняются пакетами, а изменения фиксируются короткими транзакциями. Это снижает риск длительной блокировки базы данных во время переиндексации.

2.7 Алгоритм разбиения документации на фрагменты

Разбиение документации на фрагменты выполняется после очистки HTML и выделения блоков. Парсер удаляет служебные элементы страницы, навигацию, заголовки сайта, формы, сценарии и другие шумовые элементы. Затем из содержательной части извлекаются заголовки, абзацы, элементы списков, блоки кода и таблицы. Для каждого блока сохраняется путь раздела.

Текущая реализация разбиения является символьной: используется ограничение `CHUNK_SIZE=1200` и перекрытие `CHUNK_OVERLAP=200`. Это важно явно указать, поскольку в модели данных поле называется `token_count`, но граница фрагмента задается не токенизатором модели, а длиной строки. При создании фрагмента дополнительно вычисляется приблизительное число токенов регулярным выражением.

Правила разбиения приведены в таблице 10. Они направлены на снижение смыслового шума и сохранение связи фрагмента с разделом документации.

Таблица 10 – Правила разбиения документации на фрагменты

Правило	Назначение
Не смешивать разные пути разделов	Фрагмент должен относиться к одному разделу документации, чтобы источник был понятен пользователю.
Не смешивать разные типы содержимого	Абзацы, таблицы и блоки кода не объединяются в один фрагмент при смене типа блока.
Ограничивать длину фрагмента	Фрагмент должен быть достаточно компактным для векторизации и передачи в контекст генерации.
Использовать перекрытие	Часть предыдущего текста переносится в следующий фрагмент при переполнении, чтобы не потерять связность.
Назначать педагогическую роль	Фрагменты получают роль <code>overview</code> , <code>prerequisite</code> , <code>step</code> , <code>example</code> или <code>warning</code> .

2.8 Механизм поиска фрагментов

Поиск фрагментов строится как комбинирование семантической и лексической веток. Семантическая ветка использует расстояние между вектор-

ным представлением вопроса и векторными представлениями фрагментов в pgvector. Лексическая ветка ищет совпадения расширенных терминов в названии документа, пути раздела и тексте фрагмента. Такой подход учитывает как свободные формулировки пользователя, так и точные технические термины.

На первом этапе выбираются кандидаты по версии PostgreSQL и типу корпуса. Затем кандидаты двух веток объединяются по идентификатору фрагмента. Для предварительного ранжирования используется формула:

$$pre_rank = 0,74 \cdot semantic_similarity + 0,26 \cdot lexical_signal.$$

Здесь *semantic_similarity* вычисляется из косинусного расстояния, а *lexical_signal* нормирует лексическую оценку совпадений. Формула не является итоговой оценкой качества ответа; она используется только для предварительного отбора кандидатов перед повторным ранжированием.

2.9 Учет версии PostgreSQL при поиске

Контроль версии реализуется двумя слоями. Первый слой – SQL-фильтрация по `Version.major_version`. Она не позволяет выбрать документ, связанный с другой записью версии. Второй слой – постфильтрация официальных источников по URL. Для официального корпуса допускаются только источники, содержащие ожидаемый фрагмент `/docs/<pg_version>/`. Например, для PostgreSQL 16 источник `/docs/17/sql-select.html` и источник `/docs/current/sql-select.html` должны быть исключены.

Дополнительный корпус также привязан к версии через таблицу `documents`, но его URL может иметь схему `supplementary://` или внешний источник из обработанного markdown-корпуса. Поэтому URL-проверка применяется имен-

но к официальным источникам, а базовая версионная изоляция обеспечивается связью документа с записью версии.

Проектный смысл контроля версии состоит не в гарантии абсолютной правильности ответа, а в снижении риска смешения фрагментов разных веток документации. Оставшиеся риски связаны с полнотой корпуса и качеством найденных фрагментов, поэтому после поиска выполняется повторное ранжирование и проверка подтверждений.

2.10 Повторное ранжирование найденных фрагментов

Повторное ранжирование корректирует порядок кандидатов после первичного поиска. В отличие от предварительной оценки, оно учитывает больше признаков: семантическое сходство, лексическое пересечение, совпадение с заголовком и путем раздела, совпадение технических терминов, лексический сигнал, тип корпуса, педагогическую роль и специальные корректировки для тем логической репликации, конфигурации, совместимости и сравнительных запросов.

Состав признаков приведен в таблице 11. Приоритет официального корпуса заложен явно: для краткого (short) и подробного (detailed) режимов выбираются только официальные кандидаты, а для обучающего режима (tutorial) дополнительный корпус может занять ограниченную часть контекста.

Таблица 11 – Сигналы повторного ранжирования

Сигнал	Назначение
semantic_similarity	Оценивает близость вопроса и фрагмента по векторному расстоянию.
lexical_overlap	Учитывает пересечение токенов вопроса и текста фрагмента.
title_section_overlap	Повышает оценку фрагментов, где запрос совпадает с заголовком или путем раздела.

Продолжение таблицы 11

Сигнал	Назначение
technical_term_overlap	Учитывает совпадение с выделенными техническими терминами.
corpus_bonus	Повышает приоритет официального корпуса и ограничивает влияние дополнительного корпуса.
role_bonus	Учитывает педагогическую роль фрагмента, особенно в обучающем режиме.
Тематические бонусы и штрафы	Корректируют оценку для логической репликации, параметров конфигурации, совместимости и сравнительных запросов.

2.11 Проверка достаточности подтверждений

Проверка достаточности подтверждений выполняется после повторного ранжирования. Ее задача – определить, можно ли передавать найденный контекст генеративной модели как основание для уверенного ответа. Логика проверки не использует внешнюю разметку качества, а опирается на признаки найденных фрагментов: наличие официальных источников, итоговую оценку, семантическое сходство, лексическое пересечение, совпадение с заголовком и технические термины.

Для вопросов по логической репликации проверка дополнительно требует тематические якоря: logical replication, publication, subscription, wal_level, max_replication_slots, max_wal_senders, publisher, subscriber. Если официального контекста нет или признаки слишком слабые, система возвращает безопасный отказ. Для текстовых режимов это сообщение о недостатке подтвержденных данных, а для обучающего режима – структура с пустыми шагами и пояснением ограничения.

2.12 Обработка ошибок и безопасные сценарии

Проектирование обработки ошибок опирается на принцип явного и проверяемого отказа. Если система не может корректно выполнить запрос, пользова-

тель должен получить понятную реакцию, а история запроса должна фиксировать статус. Основные ситуации приведены в таблице 12.

Таблица 12 – Обработка ошибок и безопасные сценарии

Ситуация	Причина	Реакция системы
Неподдерживаемая версия	pg_version не входит в список 16,17,18.	Ошибка валидации или отказ при разрешении версии.
Недопустимый режим ответа	Значение answer_mode не равно short, detailed или tutorial.	Ошибка валидации Pydantic.
Слишком короткий вопрос	Длина очищенного вопроса меньше трех символов.	Ошибка валидации.
Нет релевантных фрагментов	Поиск не вернул кандидатов выбранной версии.	Ошибка доменного уровня с кодом 404.
Нет официальных источников	После ранжирования остались только дополнительные источники.	Отказ с кодом 422.
Недостаточно подтверждений	Признаки релевантности официального контекста слишком слабые.	Безопасный отказ без уверенной генерации.
Недоступен Groq API	Нет ключа, неверная модель, ошибка сети или ответ сервиса.	Ошибка доменного уровня с кодом 503 и сохранение неуспешной записи истории.
Ошибка переиндексации	Сбой загрузки, векторизации или сохранения.	Откат транзакции текущего пакета и диагностическое сообщение.

2.13 Вывод по главе

Во второй главе спроектирована архитектура веб-приложения, включающая офлайн-подготовку корпуса и онлайн-обработку запроса. Описаны серверная часть, маршруты API, диаграмма последовательности, диаграмма процесса, модель данных, алгоритмы подготовки корпуса, разбиения документации на фрагменты, поиска, учета версии, повторного ранжирования и проверки достаточности подтверждений. Проектные решения согласованы с фактической реализацией и не предполагают функций, отсутствующих в программной части.

3 РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ

3.1 Выбор технологического стека

Практическая часть реализована на Python 3.12. Серверная часть построена на FastAPI, схемы входных и выходных данных описаны Pydantic, доступ к базе данных выполнен через SQLAlchemy, миграции управляются Alembic [57, 38, 39, 40, 41]. В качестве базы данных используется PostgreSQL с расширением pgvector, что позволяет хранить обычные сущности приложения и векторные представления фрагментов в одной СУБД [1, 42].

Для подготовки корпуса используется Beautiful Soup и локальные файлы HTML/Markdown [43]. Для HTTP-взаимодействия с Groq API используется httpx [44, 45]. Векторные представления в рабочем режиме формируются локальной моделью BAAI/bge-m3 через библиотеку sentence-transformers; тестовый хеширующий модуль разрешен только при APP_ENV=test [58]. Автоматические проверки реализованы на pytest [46]. Локальное развертывание описано через Docker Compose.

Выбор стека соответствует задачам ВКР: FastAPI обеспечивает маршруты API и HTML-страницы, PostgreSQL хранит корпус и историю, pgvector обеспечивает векторный поиск, Alembic фиксирует физическую схему, а pytest позволяет повторяемо проверить критичные правила.

3.2 Структура программного проекта

Структура программной части разделена по слоям. Сокращенное представление приведено в таблице 13. В таблицу включены только основные зоны ответственности, поскольку подробное перечисление внутренних файлов не несет самостоятельной инженерной ценности для отчета.

Таблица 13 – Структура программного проекта

Путь	Назначение
app/main.py	Создание FastAPI-приложения, подключение маршрутов, статических файлов, шаблонов и начального заполнения версий.
app/api/routes	Маршруты пользовательского запроса, версий, истории, диагностики и административной переиндексации.
app/schemas	Pydantic-схемы запросов, ответов, истории, версий, диагностики и административных операций.
app/db	ORM-модели, перечисления и создание сессии базы данных.
app/repositories	SQL-запросы для поиска фрагментов, индексации, истории и версий.
app/services	Обработка пользовательского запроса, анализ вопроса, поиск фрагментов, повторное ранжирование, история и администрирование.
app/services/ingestion	Загрузка, очистка, разбиение и индексация официального и дополнительного корпусов.
app/services/adapters	Адаптеры векторных представлений и генерации через Groq API.
app/web	Шаблоны HTML, стили и JavaScript пользовательского интерфейса.
tests	Автоматические модульные, API, сценарные и интеграционные тесты.

3.3 Реализация модели базы данных

Модель базы данных реализована в `app/db/models.py`. В ней определены таблицы `versions`, `documents`, `chunks`, `embeddings`, `query_history` и `query_sources`. Набор допустимых технических значений вынесен в перечисления: `official`, `supplementary`, `short`, `detailed`, `tutorial`, `success`, `failed`.

Ключевой фрагмент ORM-моделей приведен в листинге 3.1. Листинг показывает только поля, которые доказывают наличие версионной привязки, фрагментов, векторов и истории.

Листинг 3.1 – Фрагмент ORM-моделей базы данных

```

1 class Version(Base):
2     __tablename__ = "versions"
3     id = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)

```

```

4  major_version = mapped_column(String(10), unique=True, nullable=False)
5  docs_base_url = mapped_column(String(255), nullable=False)
6  is_supported = mapped_column(Boolean, nullable=False, default=True)

7  class Document(Base):
8      __tablename__ = "documents"
9      version_id = mapped_column(UUID(as_uuid=True), ForeignKey("versions.id",
    ↪      ondelete="CASCADE"), nullable=False)
10     title = mapped_column(String(500), nullable=False)
11     source_url = mapped_column(String(700), nullable=False)
12     corpus_type = mapped_column(String(30), nullable=False)

13 class Chunk(Base):
14     __tablename__ = "chunks"
15     document_id = mapped_column(UUID(as_uuid=True), ForeignKey("documents.id",
    ↪     ondelete="CASCADE"), nullable=False)
16     section_path = mapped_column(String(700), nullable=False)
17     chunk_text = mapped_column(Text, nullable=False)
18     pedagogical_role = mapped_column(String(30), nullable=False, default="overview")

19 class Embedding(Base):
20     __tablename__ = "embeddings"
21     chunk_id = mapped_column(UUID(as_uuid=True), ForeignKey("chunks.id",
    ↪     ondelete="CASCADE"), nullable=False, unique=True)
22     embedding_model = mapped_column(String(120), nullable=False)
23     embedding_dimension = mapped_column(Integer, nullable=False)
24     embedding = mapped_column(Vector(settings.embedding_dimension), nullable=False)

```

История запросов хранится отдельно от корпуса. Таблица `query_history` содержит исходный вопрос, выбранную версию, режим, текстовый ответ, JSON-структуру обучающего результата, статус и задержку. Таблица `query_sources` сохраняет использованные источники: позицию, оценку, роль, заголовок, URL, путь раздела и тип корпуса.

3.4 Реализация миграций и физической схемы PostgreSQL

Физическая схема создается миграциями Alembic. Начальная миграция создает расширение `vector`, таблицы корпуса и истории, ограничения допустимых значений, индексы и векторный индекс `ivfflat`. Миграция от 4 мая 2026 года переводит векторное поле на `vector(1024)` для модели BAAI/bge-m3. Миграции от 5 и 7 мая 2026 года фиксируют актуальную схему режима истории: технические значения `short`, `detailed`, `tutorial` и отсутствие устаревшего поля `extended_mode`.

Ключевые строки миграции показаны в листинге 3.2. Они подтверждают использование PostgreSQL, `pgvector`, `JSONB` и индекса для векторного поиска. Листинг 3.2 – Ключевые элементы миграции PostgreSQL

```
1 embedding_dim = int(os.getenv("EMBEDDING_DIMENSION", "1024"))
2 op.execute("CREATE EXTENSION IF NOT EXISTS vector")

3 op.create_table(
4     "embeddings",
5     sa.Column("chunk_id", postgresql.UUID(as_uuid=True), nullable=False, unique=True),
6     sa.Column("embedding_model", sa.String(length=120), nullable=False),
7     sa.Column("embedding_dimension", sa.Integer(), nullable=False),
8     sa.Column("embedding", Vector(embedding_dim), nullable=False),
9 )

10 op.create_table(
11     "query_history",
12     sa.Column("mode", sa.String(length=30), nullable=False),
13     sa.Column("tutorial_json", postgresql.JSONB(astext_type=sa.Text()), nullable=True),
14 )

15 op.execute(
16     "CREATE INDEX IF NOT EXISTS ix_embeddings_embedding ON embeddings "
17     "USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100)"
18 )
```

В Docker Compose используется образ `pgvector/pgvector:pg16`. Это означает, что контейнерная среда БД основана на PostgreSQL 16 с установленным расширением `pgvector`. Поддерживаемые версии PostgreSQL 16, 17 и 18 относятся к версиям документации, по которым строится корпус, а не к версии серверного контейнера базы данных.

3.5 Реализация маршрутов программного интерфейса

Маршруты API подключаются через общий маршрутизатор с префиксом `/api`. Реальная реализация содержит маршруты `/api/ask`, `/api/versions`, `/api/history`, `/api/health`, `/api/health/embeddings` и `/api/admin/reindex`. Дополнительно в `app/main.py` реализованы HTML-страницы `/` и `/history`, а также диагностические маршруты `/health` и `/health/embeddings` без префикса `/api`.

Контракт запроса `/api/ask` задается схемой `AskRequest`. Фрагмент схемы приведен в листинге 3.3. Он показывает реальные режимы ответа и обязательную валидацию версии.

Листинг 3.3 – Фрагмент схемы пользовательского запроса

```
1 class AskRequest(BaseModel):
2     model_config = ConfigDict(extra="ignore")
3
4     question: str = Field(min_length=3, max_length=4000)
5     pg_version: str = Field(min_length=1, max_length=10)
6     answer_mode: Literal["short", "detailed", "tutorial"]
7
8     @field_validator("pg_version")
9     @classmethod
10    def validate_version(cls, value: str) -> str:
11        normalized = value.strip()
12        if not normalized.isdigit():
13            raise ValueError("pg_version must be a major numeric version, e.g. '16'")
14        if normalized not in settings.supported_versions:
```

```
13         raise ValueError("pg_version must be one of supported versions")
14     return normalized
```

Схема ответа AskResponse содержит поля answer, tutorial и sources. Для текстовых режимов используется поле answer, для обучающего режима – структура tutorial.

3.6 Реализация подготовки и индексации корпуса

Официальный корпус загружается из локальной папки corpus/postgres/html/<version>. Загрузчик выбирает файлы .html и .htm, строит канонический source_url на основе OFFICIAL_DOCS_BASE_URL и выбранной версии, а при ограничении max_pages применяет приоритет для страниц логической репликации, конфигурации, SQL-команд и примечаний к выпускам.

Парсер очищает HTML от навигации, служебных блоков, форм и сценариев, затем извлекает заголовки, абзацы, списки, таблицы и блоки кода. Разбиение выполняется классом TextChunker: фрагменты ограничиваются CHUNK_SIZE=1200, используют CHUNK_OVERLAP=200, не смешивают разные section_path и content_type, а также получают педагогическую роль.

Дополнительный корпус загружается из corpus/tutorial. Индексируются markdown-файлы curated/<version> и обработанные markdown-файлы processed_html с признаком indexable. Служебные отчеты, манифесты, резервные копии, скрипты и файлы кэша не индексируются. Для curated-документов проверяется метаданное поле pg_version; документ с другой версией пропускается.

Индексация выполняется классом IndexingPipeline. Он получает или создает запись версии, подготавливает документы, вычисляет векторные представления пакетами, удаляет старый корпус выбранного типа и сохраняет новые документы, фрагменты и векторы.

В рабочем режиме используется `EMBEDDING_PROVIDER=local`, `EMBEDDING_MODEL=BAAI/bge-m3`, `EMBEDDING_DIMENSION=1024`, `EMBEDDING_MAX_SEQ_LENGTH=8192`.

3.7 Реализация обработки пользовательского запроса

Обработка запроса реализована в `AskOrchestrationService`. Сервис получает объект `AskRequest`, проверяет существование версии, анализирует вопрос, выбирает корпуса, получает векторное представление вопроса, запускает поиск фрагментов, повторное ранжирование, проверку подтверждений и генерацию. После обработки сервис сохраняет историю запроса и использованные источники.

Анализ вопроса реализован в `app/services/query_processing.py`. Он нормализует текст, определяет намерение `factual`, `procedural`, `comparative`, `compatibility`, выделяет технические термины и формирует расширенные термины для лексического поиска. Для темы логической репликации автоматически добавляются термины `logical replication`, `publication`, `subscription`, `replication slot`, `wal_level`, `max_replication_slots`, `max_wal_senders`.

Выбор корпусов зафиксирован статическим методом `resolve_corpora`. Для краткого (`short`) и подробного (`detailed`) режимов возвращается только официальный корпус (`official`); для обучающего режима (`tutorial`) возвращаются официальный и дополнительный корпуса (`official`, `supplementary`). Если обучающий поиск не вернул дополнительных кандидатов, сервис отдельно пытается получить их из дополнительного корпуса, но только после основного поиска официального корпуса.

3.8 Реализация поиска и учета версии

Поиск фрагментов реализован в `RetrievalRepository` и `RetrievalService`. Репозиторий извлекает кандидаты двумя ветками: векторной и лексической. Обе

ветки фильтруют документы по `Version.major_version` и списку допустимых типов корпуса. Векторная ветка сортирует результаты по `cosine_distance`; лексическая ветка учитывает совпадения в `title`, `section_path` и `chunk_text`.

После объединения кандидатов применяется предварительная оценка, показанная в листинге 3.4. Она используется только для сортировки кандидатов перед повторным ранжированием.

Листинг 3.4 – Формула предварительного ранжирования кандидатов

```
1 semantic_similarity = max(0.0, 1.0 - (item.distance / 2.0))
2 lexical_signal = min(item.lexical_score / 8.0, 1.0)
3 pre_rank = 0.74 * semantic_similarity + 0.26 * lexical_signal
```

Контроль версии реализован методом `_enforce_version_guard` класса `RetrievalService`. Сервис удаляет официальный источник, если в его URL нет ожидаемого фрагмента `/docs/<pg_version>/`. Тем самым SQL-фильтр по версии дополняется явной проверкой адреса официальной документации.

3.9 Реализация повторного ранжирования и проверки подтверждений

Повторное ранжирование реализовано в `RerankingService`. Для каждого кандидата вычисляется итоговая оценка на основе семантического сходства, лексического пересечения, совпадения с заголовком и путем раздела, совпадения технических терминов, лексического сигнала, типа корпуса и педагогической роли. Для логической репликации, вопросов совместимости и параметров конфигурации применяются дополнительные бонусы и штрафы.

В кратком (`short`) и подробном (`detailed`) режимах итоговый список строится только из официальных фрагментов. В обучающем режиме (`tutorial`) дополнительный корпус может быть добавлен к официальным источникам, но не заменяет их. Практическая реализация ограничивает число фрагментов одного документа, чтобы контекст не состоял из повторов одной страницы.

Проверка достаточности подтверждений реализована методом `_has_sufficient_evidence`. Она требует наличия официальных источников и проверяет пороги итоговой оценки, семантического сходства, лексического пересечения, совпадения заголовка и технических терминов. Для вопросов по логической репликации дополнительно проверяются тематические якоря. Если проверка не пройдена, генерация уверенного ответа блокируется, а пользователь получает безопасную формулировку о недостатке подтвержденных данных.

3.10 Реализация пользовательского интерфейса

Пользовательский интерфейс создан шаблонами `app/web/templates/index.html` и `app/web/templates/history.html`, стилями `app/web/static/style.css` и сценариями `app/web/static/app.js`, `app/web/static/history.js`. Главная страница содержит поле вопроса, быстрые примеры, выбор версии PostgreSQL, сегмент выбора режима ответа, блок результата, блок источников и список недавних запросов.

Структура главной страницы подчинена основному сценарию: пользователь формулирует вопрос, выбирает версию PostgreSQL и режим ответа, после чего получает результат в той же рабочей области. Интерфейс не содержит отдельного переключателя корпуса. Это согласуется с серверной логикой: состав корпуса определяется режимом ответа, а не произвольным выбором пользователя. Такой вариант снижает риск ошибки, когда пользователь случайно включает дополнительный корпус для краткого ответа и получает менее проверяемый результат.

JavaScript главной страницы загружает список версий через `/api/versions`, формирует структуру запроса для `/api/ask`, обрабатывает ответы трех режимов и отображает источники. Для краткого и подробного режимов используется текстовый блок ответа. Для обучающего режима отображаются от-

дельные секции: краткое объяснение, предварительные условия, пошаговые действия, замечания и частые ошибки. Пустые секции скрываются, поэтому пользователь не видит технических заготовок, если сервер вернул неполную обучающую структуру.

Блок загрузки показывает смену этапов обработки: поиск источников, формирование ответа и проверку формулировки. Это не добавляет новых функций, но делает длительный запрос понятнее для пользователя. При ошибке интерфейс выводит нейтральное сообщение и сохраняет технические детали в раскрываемом блоке. Такой прием отделяет пользовательское объяснение от диагностической информации: обычный пользователь видит причину сбоя в краткой форме, а разработчик может скопировать детали для отладки.

Источники отображаются отдельными карточками. Для каждого источника показываются название, путь раздела, тип корпуса, версия, ссылка и метаданные релевантности. Если источников много, интерфейс сначала показывает ограниченное число карточек и дает возможность раскрыть остальные. Это важно для читаемости: ответ остается в фокусе, но трассировка до документов не скрывается.

Страница истории обращается к `/api/history?limit=100` и показывает вопрос, дату, версию, режим, статус, краткий предварительный просмотр ответа и раскрываемый список источников. На главной странице боковая панель истории запрашивает только четыре последних запроса через `/api/history?limit=4`. Поэтому пользователь может быстро вернуться к недавнему вопросу, а полный просмотр истории вынесен на отдельную страницу.

3.11 Вывод по главе

В третьей главе описана реализация веб-приложения: выбранный стек, структура проекта, ORM-модели, миграции PostgreSQL, маршруты программного интерфейса, подготовка и индексация корпуса, обработка пользовательского

запроса, поиск фрагментов с учетом версии, повторное ранжирование, проверка достаточности подтверждений и пользовательский интерфейс. Раздел тестирования вынесен в отдельную четвертую главу, поэтому вывод по главе относится только к программной части.

4 ТЕСТИРОВАНИЕ И ОЦЕНКА РЕЗУЛЬТАТОВ

4.1 Цель и программа испытаний

Цель тестирования состоит в проверке того, что веб-приложение выполняет заявленные функции и не выходит за пределы программной реализации. Проверка ориентирована на функциональные и сценарные свойства: корректность маршрутов API, валидацию версии и режима ответа, правила выбора корпуса, поиск фрагментов, контроль версии, повторное ранжирование, проверку достаточности подтверждений, сохранение истории и работу пользовательского интерфейса.

Программа испытаний построена в соответствии с требованиями, сформулированными в первой главе, и проектными решениями второй главы. Она включает автоматические тесты, сценарные проверки пользовательских функций и сопоставление результатов с требованиями. Количественные метрики качества ранжирования, такие как Recall@K, MRR, nDCG или ассигасу, в работе не рассчитывались, поскольку для них требуется отдельный размеченный набор запросов и эталонных фрагментов [15, 32, 33].

Общая программа испытаний приведена в таблице 14. Каждый пункт связан с реализованными файлами практической части и не предполагает ручного изменения состояния корпуса или рабочей базы данных во время проверки отчета.

Таблица 14 – Программа испытаний веб-приложения

№	Направление проверки	Проверяемые свойства	Средство проверки
1	Маршруты API	Доступность диагностики, версий, пользовательского запроса и истории.	tests/test_api.py, tests/test_history_api.py
2	Валидация входных данных	Неподдерживаемая версия, недопустимый режим ответа, короткий вопрос.	Pydantic-схемы и API-тесты.
3	Выбор корпуса	Официальный корпус для краткого и подробного режимов; оба корпуса для обучающего режима.	tests/test_orchestration_rules.py, tests/test_reranking.py
4	Подготовка корпуса	Загрузка локального HTML, обработка markdown-документов, разбиение на фрагменты и проверка покрытия локального официального корпуса.	tests/test_official_loader.py, tests/test_official_corpus_coverage.py, tests/test_parser.py, tests/test_chunker_quality.py
5	Контроль версии	Исключение официальных источников другой версии и ссылки /docs/current/.	tests/test_retrieval_guard.py
6	Повторное ранжирование	Приоритет официального корпуса и тематических якорей, ограничение дополнительного корпуса.	tests/test_reranking.py.
7	Проверка подтверждений	Безопасный отказ при слабом официальном контексте.	tests/test_groundedness_guard.py
8	Генерация и стабилизация результата	Удаление источников из текста ответа, стабилизация обучающей структуры, детерминированные SQL-сценарии.	tests/test_generation_safety.py
9	Пользовательский интерфейс	Три режима ответа, отсутствие переключателя корпуса, история, SQL-блоки.	tests/test_frontend_answer_modes.py

4.2 Критерии проверки

Критерии проверки определяются не абсолютной точностью ответа, а соответствием поведения системы требованиям. Такой выбор связан с характером ВКР: практическая часть реализует инженерный прототип, а не проводит масштабное экспериментальное сравнение поисковых моделей. Критерии перечислены в таблице 15.

Таблица 15 – Критерии проверки результатов

Критерий	Условие выполнения
Корректность маршрутов API	Реализованные маршруты возвращают структуры данных, соответствующие Pydantic-схемам.
Корректность версии	Неподдерживаемые версии отклоняются, а официальные источники другой версии исключаются из результата.
Корректность режимов	Краткий (short) и подробный (detailed) режимы используют официальный корпус; обучающий режим (tutorial) допускает дополнительный корпус как вспомогательный.
Трассируемость	Результат содержит список источников или безопасный отказ при недостатке подтверждений.
Безопасный отказ	Система не формирует уверенный ответ при слабом официальном контексте.
История запросов	История содержит вопрос, версию, режим, статус, задержку, результат и источники.
Интерфейс	Главная страница поддерживает ввод вопроса, выбор версии, три режима ответа, источники и недавнюю историю.

4.3 Среда тестирования

Проверка выполнялась в директории практической части /home/arz/Desktop/RAG_postgres. Для сохранения режима чтения при запуске тестов использовались переменная `PYTHONDONTWRITEBYTECODE=1` и отключение поставщика кэша `pytest`. Это не меняет смысл команды тестирования, но предотвращает создание новых файлов `bytecode` и кэша из-за самого запуска.

Среда тестирования соответствует зависимостям проекта: Python 3.12, FastAPI 0.116.1, SQLAlchemy 2.0.43, Alembic 1.16.5, pgvector 0.4.1, Pydantic 2.11.7, httpx 0.28.1, BeautifulSoup 4.13.5, sentence-transformers 3.0.1 и pytest 8.4.2. В тестовой конфигурации автоматическая фикстура устанавливает `APP_ENV=test`, `EMBEDDING_PROVIDER=hashing`, `EMBEDDING_DIMENSION=256`, чтобы тесты не зависели от загрузки реальной модели построения векторных представлений.

Контейнерная среда проекта содержит сервисы `postgres`, `postgres_test` и сервис серверной части `backend`. Основной контейнер базы данных использует образ `pgvector/pgvector:pg16`. Интеграционные тесты рассчитаны на отдельную тестовую базу данных и пропускаются, если безопасная переменная `TEST_DATABASE_URL` не задана.

4.4 Автоматические тесты

Автоматические тесты размещены в каталоге `tests`. В проекте выделены модульные, API, сценарные и интеграционные проверки. Основной набор тестов не требует реальной рабочей базы данных и сети: маршруты API проверяются через `TestClient` и имитированные сервисы, а правила поиска и ранжирования проверяются на синтетических объектах.

Состав автоматических тестов приведен в таблице 16. Таблица отражает фактический набор файлов тестов в практической части.

Таблица 16 – Автоматические тесты практической части

Файл тестов	Проверяемая логика	Тип проверки
test_api.py	Диагностика, валидация версии и режима, краткий (short), подробный (detailed) и обучающий (tutorial) ответы, маршрут версий.	API-тесты с моками.
test_history_api.py	Возврат режима, версии и структуры истории запросов.	API-тесты.
test_frontend_answer_modes.py	Наличие трех режимов, отсутствие пользовательского переключателя корпуса, история, SQL-блоки.	Проверка HTML/JavaScript.
test_retrieval_guard.py	Исключение официальных источников другой версии и /docs/current/.	Модульные тесты.
test_reranking.py	Приоритет официального корпуса, использование дополнительного корпуса в обучающем режиме, тематические бонусы и штрафы.	Модульные тесты.
test_groundedness_guard.py	Принятие сильного официального контекста и отклонение слабого контекста.	Модульные тесты.
test_generation_safety.py	Очистка ответа от сгенерированного списка источников, стабилизация обучающего результата, детерминированные SQL-ответы.	Модульные тесты.
test_query_processing.py	Определение намерения, расширение терминов и признаки логической репликации.	Модульные тесты.
test_official_loader.py, test_official_corpus_coverage.py, test_parser.py, test_chunker_quality.py	Загрузка HTML, проверка покрытия официального корпуса, очистка документа и разбиение на фрагменты.	Модульные тесты подготовки корпуса.
test_reindex_pipeline.py, test_bge_m3_migration.py, test_provider_rules.py	Порядок переиндексации, параметры BGE-M3 и запрет хеширующего модуля вне тестов.	Модульные и сценарные тесты.

Продолжение таблицы 16

Файл тестов	Проверяемая логика	Тип проверки
tests/integration/*	Работа с реальной тестовой PostgreSQL+pgvector базой.	Интеграционные тесты, пропускаются без TEST_DATABASE_URL.

4.5 Сценарные проверки пользовательских функций

Сценарные проверки дополняют автоматические тесты и показывают, как пользовательские функции связаны между собой. Они описаны по поведению интерфейса и серверной части. Перечень сценариев приведен в таблице 17.

Таблица 17 – Сценарные проверки пользовательских функций

№	Сценарий	Входные данные	Ожидаемый результат
1	Краткий ответ	Вопрос, pg_version=16, answer_mode=short.	Возвращается поле answer и официальные источники.
2	Подробный ответ	Вопрос, pg_version=16, answer_mode=detailed.	Возвращается поле answer, источники относятся к официальному корпусу.
3	Обучающий ответ	Вопрос, pg_version=16, answer_mode=tutorial.	Возвращается объект tutorial с четырьмя секциями и источниками.
4	Недопустимый режим	answer_mode=extended.	Запрос отклоняется валидацией.
5	Неподдерживаемая версия	pg_version=19.	Запрос отклоняется как неподдерживаемый.
6	Недоступность генерации	Отсутствует GROQ_API_KEY или некорректна модель.	Возвращается ошибка сервиса, история фиксирует неуспешный запрос.
7	Просмотр истории	Запрос /api/history?limit=50.	Возвращаются записи с вопросом, версией, режимом, статусом и источниками.

После выполнения каждого сценария проверялись не только поля ответа, но и связь результата с выбранным режимом. Для краткого режима достаточно наличия текстового ответа и официальных источников выбранной версии. Для подробного режима дополнительно важна полнота объяснения в рамках найденного контекста, но без заявления численной оценки качества. Для обучающего режима проверяется структура результата: краткое объяснение, предварительные условия, шаги и замечания. Если часть обучающих секций пуста, интерфейс скрывает ее, а не показывает пользователю техническую заготовку.

Отдельное внимание уделялось отрицательным сценариям. Неподдерживаемая версия и недопустимый режим ответа должны останавливаться до обработки запроса, поскольку дальнейший поиск в таком случае не имеет корректного корпуса или режима. Недоступность генерации проверяется как сбой внешней зависимости: история фиксирует неуспешный запрос, а пользователь получает сообщение об ошибке. Такой сценарий не подтверждает качество ответа, но подтверждает устойчивость пользовательского потока при отказе внешнего сервиса.

Сценарий просмотра истории связывает пользовательский интерфейс с сохранением результата. Проверяется, что запись содержит вопрос, версию, режим, статус и источники, а сама страница истории не требует повторного выполнения запроса. Это важно для практического использования: пользователь может вернуться к ранее полученному ответу и повторно открыть источники, не обращаясь заново к генеративной модели.

4.6 Проверка учета версии PostgreSQL

Контроль версии проверяется на двух уровнях. Первый уровень связан с валидацией `AskRequest.pg_version`: значение должно быть числовой основной версией и входить в список 16,17,18. Второй уровень связан с фильтраци-

ей источников после поиска. Тесты из `test_retrieval_guard.py` проверяют, что официальный источник `/docs/17/` исключается из результата для версии 16, а ссылка `/docs/current/` не принимается как источник версии 18.

Эти тесты подтверждают наличие контроля версии, но не утверждают абсолютную полноту поиска. Система может корректно фильтровать найденные источники, однако качество ответа по конкретному вопросу зависит от полноты локального корпуса и релевантности найденных фрагментов.

4.7 Проверка безопасного отказа при недостатке подтверждений

Проверка достаточности подтверждений тестируется на слабом и сильном контексте. В слабом сценарии вопрос относится к логической репликации, но найденный фрагмент связан с другой темой. Тест подтверждает, что метод `_has_sufficient_evidence` возвращает `False`. В сильном сценарии официальный фрагмент содержит понятия `Logical Replication`, `publication` и `subscription`, поэтому проверка возвращает `True`.

В пользовательском сценарии отрицательный результат проверки приводит не к вызову генеративной модели, а к безопасному отказу. Для краткого (`short`) и подробного (`detailed`) режимов возвращается текст о недостатке подтвержденных данных в официальной документации выбранной версии. Для обучающего режима (`tutorial`) возвращается структура с пустым списком шагов и поясняющими замечаниями.

4.8 Проверка интерфейса и истории запросов

Пользовательский интерфейс проверяется статическими тестами HTML и JavaScript. Тесты подтверждают, что на главной странице есть только три режима ответа: «Краткий», «Подробный» и «Обучающий». Отдельный пользовательский переключатель дополнительного корпуса отсутствует. JavaScript формирует структуру запроса из `question`, `pg_version`, `answer_mode`.

Также проверены скрытие пустых секций обучающего ответа, кнопка копирования SQL-блока, блок технических деталей ошибки, ограничение боковой истории четырьмя записями и нейтральный статус завершения на странице истории. API истории проверяет наличие режима и версии в ответе.

4.9 Результаты запуска тестов

Фактический запуск тестов выполнен в практической части следующей командой:

```
1 PYTHONDONTWRITEBYTECODE=1 .venv/bin/python -m pytest -q -p no:cacheprovider
```

Ключевой результат запуска:

91 passed, 3 skipped in 1.00s

Три теста пропущены, поскольку интеграционные тесты требуют безопасно заданную переменную `TEST_DATABASE_URL` и отдельную тестовую PostgreSQL+pgvector базу. Это поведение описано в `tests/README.md`: если переменная не задана, интеграционные тесты не выполняются. Такой пропуск не является ошибкой автоматического набора, а защищает рабочую базу данных от случайного использования в тестах.

4.10 Сопоставление результатов с требованиями

Сопоставление требований и проверок приведено в таблице 18. Таблица показывает, какие тесты или сценарии подтверждают заявленные функции. Если требование проверяется не одним тестом, а группой проверок, указаны основные файлы и сценарии.

Таблица 18 – Сопоставление требований и тестов

ID	Требование	Проверяющие тесты и сценарии	Результат
FR-01	Поддержка версий PostgreSQL	test_api.py, маршрут /api/versions.	Подтверждено.
FR-02	Подготовка официального корпуса	test_official_loader.py, test_official_corpus_coverage.py.	Подтверждено.
FR-03	Разбиение документации на фрагменты	test_parser.py, test_chunker_quality.py.	Подтверждено.
FR-04	Векторные представления и размерность	test_bge_m3_migration.py, test_provider_rules.py, /api/health/embeddings.	Подтверждено.
FR-05	Поиск с учетом версии	test_retrieval_guard.py, интеграционный тест при наличии тестовой БД.	Частично подтверждено unit-тестами; интеграционные проверки пропущены без TEST_DATABASE_URL.
FR-06	Повторное ранжирование	test_reranking.py.	Подтверждено.
FR-07	Краткий режим (short)	test_api.py, test_reranking.py.	Подтверждено.
FR-08	Подробный режим (detailed)	test_api.py, test_reranking.py.	Подтверждено.
FR-09	Обучающий режим (tutorial)	test_api.py, test_generation_safety.py, test_frontend_answer_modes.py.	Подтверждено.
FR-10	Проверка подтверждений	test_groundedness_guard.py.	Подтверждено.
FR-11	История запросов	test_history_api.py, интеграционный тест истории при наличии тестовой БД.	Подтверждено на уровне API; интеграционная проверка пропущена без TEST_DATABASE_URL.
FR-12	Служебная диагностика	test_api.py.	Подтверждено.
FR-13	Административная переиндексация	test_reindex_pipeline.py, интеграционный тест переиндексации при наличии тестовой БД.	Подтверждено unit-тестом; интеграционная проверка пропущена без TEST_DATABASE_URL.
FR-14	Пользовательский интерфейс	test_frontend_answer_modes.py.	Подтверждено.

Продолжение таблицы 18

ID	Требование	Проверяющие тесты и сценарии	Результат
NFR-01–NFR-08	Нефункциональные требования	Совокупность тестов API, интерфейса, контроля версии, истории, поставщиков векторных представлений и проверки подтверждений.	Подтверждено функционально и сценарно.

4.11 Ограничения разработанного решения

Разработанное приложение является инженерным прототипом, а не завершённой промышленной платформой. Основные ограничения связаны с полнотой корпуса, доступностью внешней генеративной модели и методикой оценки. Поддерживаются только версии PostgreSQL, заданные в конфигурации и обеспеченные локальным корпусом: 16, 17 и 18. Добавление новой версии требует наличия корпуса HTML-документации, записи версии и переиндексации.

Качество поиска фрагментов зависит от полноты локальных HTML-страниц, корректности очистки документации, параметров разбиения и качества векторных представлений. В работе не рассчитаны количественные метрики качества ранжирования, потому что не подготовлен эталонный набор запросов и релевантных фрагментов. Поэтому выводы о качестве ограничены функциональными и сценарными проверками.

Генерация итогового текста зависит от доступности Groq API, корректности `GROQ_API_KEY` и выбранной модели `LLM_MODEL`. При недоступности генерации система возвращает диагностическую ошибку, но не может сформировать новый текстовый ответ. При этом часть детерминированных SQL-сценариев стабилизирована в коде и проверяется автоматическими тестами.

4.12 Вывод по главе

В четвертой главе определены цель, программа и критерии испытаний, описана среда тестирования, приведены автоматические и сценарные проверки, проверены учет версии PostgreSQL, безопасный отказ, пользовательский интерфейс и история запросов. Фактический запуск тестов завершился результатом 91 passed, 3 skipped. Сопоставление с требованиями показало, что заявленные функции подтверждены автоматическими или сценарными

проверками, а ограничения решения связаны с отсутствием количественной оценки ранжирования, зависимостью от полноты корпуса и доступностью генеративной модели.

ЗАКЛЮЧЕНИЕ

В выпускной квалификационной работе разработано веб-приложение на основе RAG для работы с документацией PostgreSQL с учетом выбранной версии. Реализация ориентирована на инженерный сценарий: ответ должен быть связан с локально индексированным корпусом, выбранной основной версией PostgreSQL и списком источников.

В первой главе выполнен анализ предметной области, особенностей официальной документации PostgreSQL, проблемы учета версии и существующих подходов поиска по технической документации. Сравнение показало, что обычный поиск, полнотекстовый поиск, поиск в локальном корпусе HTML-документации и вопросно-ответные системы без источников не обеспечивают одновременно связный ответ, контроль версии, локальную управляемость корпуса и трассируемость источников.

Во второй главе спроектировано веб-приложение: определены архитектура серверной части, маршруты API, сценарий обработки запроса, диаграмма процесса, модель данных, алгоритмы подготовки корпуса, разбиения документации на фрагменты, поиска, учета версии, повторного ранжирования и проверки достаточности подтверждений.

В третьей главе описана программная реализация: FastAPI-приложение, PostgreSQL с pgvector, SQLAlchemy-модели, Alembic-миграции, Pydantic-схемы, подготовка официального и дополнительного корпусов, обработка пользовательского запроса, работа с Groq API, история запросов и пользовательский интерфейс. Приложение поддерживает версии PostgreSQL 16, 17 и 18, краткий (short), подробный (detailed) и обучающий (tutorial) режимы ответа, локальную модель построения векторных представлений BAAI/bge-m3 размерности 1024 и отображение источников.

В четвертой главе проведены тестирование и оценка соответствия требованиям. Автоматический запуск тестов практической части завершился резуль-

татом 91 passed, 3 skipped. Пропущенные тесты относятся к интеграционным проверкам, требующим отдельную тестовую PostgreSQL+pgvector базу через TEST_DATABASE_URL. Проведенная проверка подтвердила работу маршрутов API, валидации версии и режима, контроля версии, повторного ранжирования, проверки достаточности подтверждений, пользовательского интерфейса и истории запросов.

Оценка результатов выполнена функционально и сценарно. Количественные метрики качества поиска и ранжирования в работе не рассчитывались, поэтому в выводах не заявляются Recall@K, MRR, nDCG, accuracy или абсолютная точность ответа. Ограничения разработанного решения связаны с полнотой локального корпуса, доступностью Groq API, заданным набором поддерживаемых версий и отсутствием отдельного эталонного набора для численной оценки качества ранжирования.

Поставленная цель достигнута: разработано и описано веб-приложение, которое принимает вопрос пользователя, версию PostgreSQL и режим ответа, извлекает релевантные фрагменты корпуса, учитывает выбранную версию, возвращает источники, сохраняет историю запросов и поддерживает безопасный отказ при недостатке подтвержденных данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The PostgreSQL Global Development Group. PostgreSQL Documentation. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/> (дата обращения: 10.05.2026).
2. The PostgreSQL Global Development Group. PostgreSQL 16 Documentation. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 10.05.2026).
3. The PostgreSQL Global Development Group. PostgreSQL 17 Documentation. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/17/> (дата обращения: 10.05.2026).
4. The PostgreSQL Global Development Group. PostgreSQL 18 Documentation. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/18/> (дата обращения: 10.05.2026).
5. The PostgreSQL Global Development Group. PostgreSQL Versioning Policy. – [Электронный ресурс]. – URL: <https://www.postgresql.org/support/versioning/> (дата обращения: 10.05.2026).
6. The PostgreSQL Global Development Group. PostgreSQL Release Notes Archive. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/release/> (дата обращения: 10.05.2026).
7. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / Patrick Lewis, Ethan Perez, Aleksandra Piktus et al. // Advances in Neural Information Processing Systems. – Vol. 33. – 2020. – Pp. 9459–9474.
8. Dense Passage Retrieval for Open-Domain Question Answering / Vladimir Karpukhin, Barlas Oguz, Sewon Min et al. // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. – 2020. – Pp. 6769–6781.
9. Reimers, Nils. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks / Nils Reimers, Iryna Gurevych // Proceedings of the 2019

Conference on Empirical Methods in Natural Language Processing. – 2019. – Pp. 3982–3992.

10. Khattab, Omar. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT / Omar Khattab, Matei Zaharia // Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. – 2020. – Pp. 39–48.

11. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding / Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova // Proceedings of NAACL-HLT 2019. – 2019. – Pp. 4171–4186.

12. Sommerville, Ian. Software Engineering / Ian Sommerville. – 10 edition. – Boston: Pearson, 2016.

13. Pressman, Roger S. Software Engineering: A Practitioner's Approach / Roger S. Pressman, Bruce R. Maxim. – 9 edition. – New York: McGraw-Hill Education, 2020.

14. Федеральное агентство по техническому регулированию и метрологии. ГОСТ Р ИСО/МЭК 25010-2015. Требования и оценка качества систем и программного обеспечения. Модели качества. – М.: Стандартинформ. – 2016.

15. Manning, Christopher D. Introduction to Information Retrieval / Christopher D. Manning, Prabhakar Raghavan, Hinrich Schütze. – Cambridge: Cambridge University Press, 2008.

16. Robertson, Stephen. The Probabilistic Relevance Framework: BM25 and Beyond / Stephen Robertson, Hugo Zaragoza // Foundations and Trends in Information Retrieval. – 2009. – Vol. 3, no. 4. – Pp. 333–389.

17. Malkov, Yu. A. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs / Yu. A. Malkov, D. A. Yashunin // IEEE Transactions on Pattern Analysis and Machine Intelligence. – 2020. – Vol. 42, no. 4. – Pp. 824–836.

18. Johnson, Jeff. Billion-scale similarity search with GPUs / Jeff Johnson, Matthijs Douze, Hervé Jégou // IEEE Transactions on Big Data. – 2021. – Vol. 7, no. 3. – Pp. 535–547.
19. Codd, E. F. A Relational Model of Data for Large Shared Data Banks / E. F. Codd // Communications of the ACM. – 1970. – Vol. 13, no. 6. – Pp. 377–387.
20. Ullman, Jeffrey D. Principles of Database and Knowledge-Base Systems / Jeffrey D. Ullman. – Rockville: Computer Science Press, 1988.
21. Date, C. J. An Introduction to Database Systems / C. J. Date. – 8 edition. – Boston: Pearson, 2004.
22. Garcia-Molina, Hector. Database Systems: The Complete Book / Hector Garcia-Molina, Jeffrey D. Ullman, Jennifer Widom. – 2 edition. – Upper Saddle River: Pearson, 2008.
23. Silberschatz, Abraham. Database System Concepts / Abraham Silberschatz, Henry F. Korth, S. Sudarshan. – 7 edition. – New York: McGraw-Hill Education, 2020.
24. Elmasri, Ramez. Fundamentals of Database Systems / Ramez Elmasri, Shamkant B. Navathe. – 7 edition. – Boston: Pearson, 2016.
25. Martin, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design / Robert C. Martin. – Boston: Prentice Hall, 2017.
26. Fowler, Martin. Patterns of Enterprise Application Architecture / Martin Fowler. – Boston: Addison-Wesley, 2002.
27. Fowler, Martin. Refactoring: Improving the Design of Existing Code / Martin Fowler. – 2 edition. – Boston: Addison-Wesley, 2018.
28. Bass, Len. Software Architecture in Practice / Len Bass, Paul Clements, Rick Kazman. – 4 edition. – Boston: Addison-Wesley, 2021.
29. Larman, Craig. Applying UML and Patterns / Craig Larman. – 3 edition. – Upper Saddle River: Prentice Hall, 2004.
30. Newman, Sam. Building Microservices / Sam Newman. – 2 edition. – Sebastopol: O'Reilly Media, 2021.

31. Richards, Mark. Fundamentals of Software Architecture / Mark Richards, Neal Ford. – Sebastopol: O'Reilly Media, 2020.
32. Beizer, Boris. Software Testing Techniques / Boris Beizer. – 2 edition. – New York: Van Nostrand Reinhold, 1990.
33. Myers, Glenford J. The Art of Software Testing / Glenford J. Myers, Corey Sandler, Tom Badgett. – 3 edition. – Hoboken: John Wiley and Sons, 2011.
34. Kaner, Cem. Testing Computer Software / Cem Kaner, Jack Falk, Hung Q. Nguyen. – 2 edition. – New York: John Wiley and Sons, 1999.
35. Docker, Inc. Docker Documentation. – [Электронный ресурс]. – URL: <https://docs.docker.com/> (дата обращения: 10.05.2026).
36. The PostgreSQL Global Development Group. PostgreSQL 16: Logical Replication. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/logical-replication.html> (дата обращения: 10.05.2026).
37. The PostgreSQL Global Development Group. PostgreSQL 16: Runtime Configuration – Replication. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/runtime-config-replication.html> (дата обращения: 10.05.2026).
38. Ramírez, Sebastián. FastAPI Documentation. – [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 10.05.2026).
39. Pydantic Team. Pydantic Documentation. – [Электронный ресурс]. – URL: <https://pydantic.dev/docs/> (дата обращения: 10.05.2026).
40. SQLAlchemy Authors. SQLAlchemy Documentation. – [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org/20/> (дата обращения: 10.05.2026).
41. Alembic Authors. Alembic Documentation. – [Электронный ресурс]. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 10.05.2026).
42. Kane, Andrew. pgvector: Open-source vector similarity search for PostgreSQL. – [Электронный ресурс]. – URL: <https://github.com/pgvector/pgvector> (дата обращения: 10.05.2026).

43. Richardson, Leonard. Beautiful Soup Documentation. – [Электронный ресурс]. – URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> (дата обращения: 10.05.2026).
44. Encode OSS Ltd. HTTPX Documentation. – [Электронный ресурс]. – URL: <https://www.python-httpx.org/> (дата обращения: 10.05.2026).
45. Groq, Inc. Groq API Documentation. – [Электронный ресурс]. – URL: <https://console.groq.com/docs/overview> (дата обращения: 10.05.2026).
46. pytest development team. pytest Documentation. – [Электронный ресурс]. – URL: <https://docs.pytest.org/> (дата обращения: 10.05.2026).
47. The PostgreSQL Global Development Group. PostgreSQL 16: Full Text Search. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/textsearch.html> (дата обращения: 10.05.2026).
48. Межгосударственный совет по стандартизации, метрологии и сертификации. ГОСТ 7.32-2017. СИБИБД. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ. – 2018.
49. Федеральное агентство по техническому регулированию и метрологии. ГОСТ Р 7.0.100-2018. СИБИБД. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – М.: Стандартинформ. – 2018.
50. Федеральное агентство по техническому регулированию и метрологии. ГОСТ Р 7.0.5-2008. СИБИБД. Библиографическая ссылка. Общие требования и правила составления. – М.: Стандартинформ. – 2008.
51. Госстандарт СССР. ГОСТ 19.201-78. ЕСПД. Техническое задание. Требования к содержанию и оформлению. – М.: Издательство стандартов. – 1978.
52. Госстандарт СССР. ГОСТ 34.601-90. Информационная технология. Автоматизированные системы. Стадии создания. – М.: Издательство стандартов. – 1990.

53. Госстандарт СССР. ГОСТ 34.602-89. Информационная технология. Техническое задание на создание автоматизированной системы. – М.: Издательство стандартов. – 1989.
54. Kleppmann, Martin. Designing Data-Intensive Applications / Martin Kleppmann. – Sebastopol: O'Reilly Media, 2017.
55. The PostgreSQL Global Development Group. PostgreSQL: JSON Types. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/current/datatype-json.html> (дата обращения: 10.05.2026).
56. The PostgreSQL Global Development Group. PostgreSQL: UUID Type. – [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/current/datatype-uuid.html> (дата обращения: 10.05.2026).
57. Python Software Foundation. Python 3 Documentation. – [Электронный ресурс]. – URL: <https://docs.python.org/3/> (дата обращения: 10.05.2026).
58. UKP Lab. Sentence Transformers Documentation. – [Электронный ресурс]. – URL: <https://www.sbert.net/> (дата обращения: 10.05.2026).

ПРИЛОЖЕНИЕ А

Техническое задание

Общие сведения

Настоящее техническое задание определяет требования к разработке веб-приложения на основе RAG для работы с документацией PostgreSQL с учетом версии. Техническое задание составлено для программного прототипа, реализованного в практической части выпускной квалификационной работы.

Наименование разработки: веб-приложение на основе RAG для работы с документацией PostgreSQL с учетом версии.

Основанием для разработки является задание на выпускную квалификационную работу по направлению 09.03.01 Информатика и вычислительная техника, образовательная программа «Системная и программная инженерия».

Исполнитель разработки: Жатиков Артур Русланович.

Назначение разработки

Разрабатываемое веб-приложение предназначено для поиска и формирования ответов по документации PostgreSQL с учетом выбранной основной версии СУБД. Пользователь должен иметь возможность задать вопрос, выбрать версию PostgreSQL и режим ответа, после чего система должна вернуть результат с указанием источников.

Приложение ориентировано на работу с локально подготовленным корпусом документации PostgreSQL по версиям 16, 17 и 18. В кратком и подробном режимах источником фактических сведений должен быть официальный корпус документации. В обучающем режиме допускается использование допол-

нительного учебного корпуса как вспомогательного слоя, но официальный корпус должен оставаться основой результата.

Требования к функциональным характеристикам

Система должна поддерживать следующие пользовательские функции:

1. получение списка поддерживаемых основных версий PostgreSQL;
2. ввод пользовательского вопроса через веб-интерфейс;
3. выбор версии PostgreSQL из поддерживаемого набора 16, 17 и 18;
4. выбор одного из трех режимов ответа: краткого, подробного или обучающего;
5. формирование краткого ответа по официальному корпусу документации;
6. формирование подробного ответа по официальному корпусу документации;
7. формирование обучающего результата со структурой краткого объяснения, предварительных условий, шагов и замечаний;
8. отображение использованных источников с названием, адресом, типом корпуса, путем раздела, позицией в выдаче и оценкой релевантности;
9. просмотр истории запросов и ранее сохраненных источников;
10. отображение диагностического сообщения при ошибке обработки запроса.

Система должна поддерживать следующие функции подготовки корпуса:

1. загрузку локальных HTML-страниц официальной документации PostgreSQL по версиям 16, 17 и 18;
2. загрузку дополнительного учебного корпуса из локальных markdown-файлов;
3. очистку документации от служебных элементов;

4. выделение текстовых фрагментов с сохранением заголовка, пути раздела, адреса источника, типа корпуса и версии;

5. вычисление векторных представлений фрагментов с использованием модели BAAI/bge-m3;

6. сохранение документов, фрагментов и векторных представлений в PostgreSQL с расширением pgvector;

7. административный запуск переиндексации корпуса с передачей списка версий, признаков включения корпусов и ограничения числа страниц.

Система должна поддерживать следующие функции обработки запроса:

1. валидацию вопроса, выбранной версии PostgreSQL и режима ответа;

2. отклонение неподдерживаемой версии PostgreSQL;

3. анализ вопроса, нормализацию текста и выделение технических терминов;

4. построение векторного представления пользовательского вопроса;

5. выбор допустимых корпусов по режиму ответа;

6. поиск релевантных фрагментов в PostgreSQL с использованием pgvector;

7. исключение официальных источников другой версии и ссылок вида /docs/current/;

8. повторное ранжирование найденных фрагментов по семантическим, лексическим и терминологическим признакам;

9. проверку достаточности официальных подтверждений перед генерацией уверенного ответа;

10. формирование безопасного отказа при недостатке подтвержденных данных;

11. обращение к API языковой модели Groq только после подготовки проверяемого контекста;

12. сохранение успешных и неуспешных запросов в истории.

Требования к данным и информационной совместимости

Входными данными пользовательского запроса являются текст вопроса, выбранная основная версия PostgreSQL и режим ответа. Запрос к маршруту `/api/ask` должен содержать поля `question`, `pg_version`, `answer_mode`. Допустимыми значениями режима ответа являются `short`, `detailed`, `tutorial`.

Выходными данными пользовательского запроса являются режим ответа, версия PostgreSQL, текстовый ответ или обучающая структура, а также список источников. Для краткого и подробного режимов используется поле `answer`. Для обучающего режима используется объект `tutorial` с полями `short_explanation`, `prerequisites`, `steps`, `notes`.

Данные корпуса и истории должны храниться в базе PostgreSQL. Физическая модель должна включать сущности версий, документов, фрагментов, векторных представлений, истории запросов и источников запроса. Векторные представления фрагментов должны храниться с размерностью 1024 для модели BAAI/bge-m3.

Требования к надежности и безопасности

Система должна предотвращать смешение официальных источников разных версий PostgreSQL. Официальный источник может использоваться в ответе только тогда, когда его адрес соответствует выбранной версии документации. Система не должна формировать уверенный ответ, если найденный контекст не содержит достаточного официального подтверждения. В таком случае пользователь должен получить безопасную формулировку о недостатке подтвержденных данных.

Секреты внешней языковой модели и административный ключ не должны храниться в исходном коде. Они должны передаваться через переменные

окружения. Контроль административного маршрута переиндексации должен выполняться при заданном административном ключе в конфигурации.

При недоступности API языковой модели система должна вернуть диагностическую ошибку и сохранить неуспешную запись истории, не подменяя ее неподтвержденным ответом.

Требования к программному и техническому обеспечению

Серверная часть должна быть реализована на Python 3.12 с использованием FastAPI, Pydantic, SQLAlchemy и Alembic. Хранение данных должно выполняться в PostgreSQL с расширением pgvector. Для построения векторных представлений должна использоваться локальная модель BAAI/bge-m3 через библиотеку sentence-transformers. Для генерации итогового текста должен использоваться API языковой модели Groq.

Пользовательский интерфейс должен быть реализован как веб-интерфейс на HTML, CSS и JavaScript. Интерфейс должен предоставлять главную страницу для ввода запроса и страницу просмотра истории. Взаимодействие клиентской и серверной частей должно выполняться через HTTP API.

Локальное развертывание должно поддерживаться через Docker Compose. Конфигурация приложения должна задаваться переменными окружения, включая адрес базы данных, список поддерживаемых версий, параметры модели векторных представлений и ключ API языковой модели.

Требования к программной документации

В составе выпускной квалификационной работы должна быть представлена пояснительная записка, включающая анализ предметной области, постановку задачи, требования, архитектуру веб-приложения, модель данных, алгоритмы подготовки корпуса и обработки запроса, описание реализации, поль-

зовательского интерфейса, тестирования и ограничений разработанного решения.

В приложениях должны быть представлены техническое задание, примеры запросов и ответов программного интерфейса, фрагменты листинга программного кода и дополнительные результаты тестирования.

Стадии и этапы разработки

Разработка выполняется по следующим этапам:

1. анализ предметной области и структуры документации PostgreSQL;
2. анализ проблемы учета версии PostgreSQL при формировании ответа;
3. формулирование функциональных и нефункциональных требований;
4. проектирование архитектуры веб-приложения и модели данных;
5. разработка алгоритмов подготовки корпуса, поиска, повторного ранжирования и проверки подтверждений;
6. реализация серверной части и маршрутов программного интерфейса;
7. реализация пользовательского интерфейса и страницы истории;
8. подготовка локального корпуса документации и дополнительного учебного корпуса;
9. тестирование пользовательских сценариев и критичных правил обработки запроса;
10. оформление пояснительной записки и приложений.

Порядок контроля и приемки

Контроль выполнения требований должен проводиться на основе автоматических тестов, сценарных проверок пользовательских функций и сопоставле-

ния реализации с функциональными и нефункциональными требованиями. Критичными проверками являются валидация версии и режима ответа, выбор корпуса по режиму, исключение источников другой версии, повторное ранжирование, проверка достаточности подтверждений, сохранение истории и отображение источников.

Результат считается соответствующим техническому заданию, если приложение запускается в заданной конфигурации, принимает пользовательский запрос, учитывает выбранную версию PostgreSQL, возвращает результат выбранного режима с источниками, сохраняет историю и проходит автоматические тесты, предусмотренные практической частью.

Ограничения разработки

Разработанное приложение является инженерным прототипом. Поддерживаются только версии PostgreSQL, для которых подготовлен локальный корпус и которые указаны в конфигурации. В текущей реализации это версии 16, 17 и 18. Добавление новой версии требует наличия корпуса документации и переиндексации.

Количественные метрики качества поиска и ранжирования не нормируются в настоящем техническом задании, поскольку для них требуется отдельный размеченный набор эталонных запросов и релевантных фрагментов. Оценка результата выполняется функционально и сценарно.

Трассировка требований к реализации

Задание на выпускную квалификационную работу включено в подключаемый файл `extra/Title.pdf`. В таблице 19 приведена краткая трассировка ключевых пунктов задания и настоящего технического задания к фактической реализации.

Таблица 19 – Соответствие пунктов задания фактической реализации

Пункт задания	Фактическая реализация
Хранение документации PostgreSQL по версиям	Таблица versions, локальный корпус corpus/postgres/html/16, 17, 18.
Разбиение документации на фрагменты	Парсер HTML и TextChunker с параметрами CHUNK_SIZE=1200, CHUNK_OVERLAP=200.
Векторные представления	Таблица embeddings, модель BAAI/bge-m3, размерность 1024.
Поиск и повторное ранжирование	RetrievalRepository, RetrievalService, RerankingService.
Учет выбранной версии	SQL-фильтр по Version.major_version и проверка URL официальных источников.
Режимы ответа	Схема AskRequest.answer_mode, сервис AskOrchestrationService, интерфейс с кратким (short), подробным (detailed) и обучающим (tutorial) режимами.
Проверка подтверждений	Метод проверки достаточности контекста в AskOrchestrationService; безопасный отказ при недостатке официальных источников.
Отображение источников	Схема SourceOut, таблица query_sources, блок источников в интерфейсе.
Сохранение истории	Таблицы query_history и query_sources, маршрут /api/history, страница /history.
Пользовательский интерфейс	index.html, history.html, app.js, history.js, style.css.

ПРИЛОЖЕНИЕ Б

Примеры запросов и ответов программного интерфейса

Пример запроса в кратком режиме (short)

```
POST /api/ask
Content-Type: application/json

{
  "question": "Как включить логическую репликацию?",
  "pg_version": "16",
  "answer_mode": "short"
}
```

Пример запроса в подробном режиме (detailed)

```
POST /api/ask
Content-Type: application/json

{
  "question": "Подробно объясни VACUUM в PostgreSQL",
  "pg_version": "16",
  "answer_mode": "detailed"
}
```

Пример запроса в обучающем режиме (tutorial)

```
POST /api/ask
Content-Type: application/json

{
  "question": "Объясни EXPLAIN ANALYZE простыми словами",
  "pg_version": "16",
  "answer_mode": "tutorial"
}
```

Пример ответа в кратком режиме (short)

```
{
  "answer_mode": "short",
  "pg_version": "16",
  "answer": "...",
  "sources": [
    {
      "title": "PostgreSQL 16 Documentation",
      "url": "https://www.postgresql.org/docs/16/...",
      "corpus_type": "official",
      "source_role": "base",
      "section_path": "Chapter 31 / Logical Replication",
      "rank_position": 1,
      "similarity_score": 0.91234
    }
  ]
}
```

Пример ответа в обучающем режиме (tutorial)

```

{
  "answer_mode": "tutorial",
  "pg_version": "16",
  "tutorial": {
    "short_explanation": "...",
    "prerequisites": ["..."],
    "steps": ["..."],
    "notes": ["..."]
  },
  "sources": [
    {
      "title": "PostgreSQL 16 Documentation",
      "url": "https://www.postgresql.org/docs/16/...",
      "corpus_type": "official",
      "source_role": "base",
      "section_path": "Chapter 31 / Logical Replication",
      "rank_position": 1,
      "similarity_score": 0.94211
    },
    {
      "title": "Tutorial note",
      "url": "supplementary://curated/16/guide.md",
      "corpus_type": "supplementary",
      "source_role": "supplementary",
      "section_path": "Guide / Logical Replication",
      "rank_position": 2,
      "similarity_score": 0.72111
    }
  ]
}

```

ПРИЛОЖЕНИЕ В

Руководство пользователя

Назначение пользовательского интерфейса

Пользовательский интерфейс предназначен для отправки вопросов по документации PostgreSQL, выбора версии документации и выбора режима ответа. После обработки запроса приложение показывает результат, список использованных источников и сведения о последних запросах.

Главная страница приложения доступна по маршруту /. Страница полной истории запросов доступна по маршруту /history. Отдельного переключателя корпуса в интерфейсе нет: состав корпуса определяется выбранным режимом ответа.

Получение ответа

Для получения ответа пользователь выполняет следующие действия:

1. открывает главную страницу приложения;
2. вводит вопрос по документации PostgreSQL в поле запроса;
3. выбирает поддерживаемую версию PostgreSQL: 16, 17 или 18;
4. выбирает режим ответа: краткий, подробный или обучающий;
5. отправляет запрос и ожидает завершения обработки;
6. просматривает сформированный результат и список источников.

Краткий режим предназначен для сжатого ответа по официальной документации. Подробный режим используется, когда требуется более развернутое объяснение по официальному корпусу. Обучающий режим возвращает структурированный результат: краткое объяснение, предварительные условия, пошаговые действия и замечания. В обучающем режиме дополнительный учебный корпус используется только как вспомогательный слой.

Просмотр источников

После успешной обработки запроса под ответом отображается список источников. Для каждого источника показывается название, путь раздела, тип корпуса, версия, ссылка и показатели релевантности. Если источников больше, чем помещается в начальном блоке, интерфейс позволяет раскрыть дополнительные элементы списка.

Источники нужны для ручной проверки ответа. Если система не нашла достаточного подтверждения в официальной документации выбранной версии, вместо уверенного ответа выводится безопасная формулировка о недостатке подтвержденных данных.

Просмотр истории

На главной странице отображаются последние запросы. Для полного просмотра истории пользователь переходит на страницу /history. На этой странице доступны вопрос, дата, выбранная версия, режим ответа, статус обработки, краткий предварительный просмотр результата и использованные источники.

История позволяет вернуться к ранее полученному ответу без повторного выполнения того же запроса. Если запрос завершился ошибкой, запись истории сохраняет статус неуспешной обработки и помогает понять, на каком этапе возникла проблема.

Действия при ошибке

Если выбранная версия не поддерживается, вопрос слишком короткий или режим ответа некорректен, приложение возвращает сообщение об ошибке валидации. Если недоступна внешняя языковая модель, пользователь полу-

чает диагностическое сообщение, а система не подменяет результат неподтвержденным ответом.

При ошибке пользователь должен проверить введенный вопрос, выбранную версию PostgreSQL и режим ответа. Если ошибка связана с внешней моделью или настройками окружения, требуется проверить конфигурацию приложения и наличие необходимых переменных окружения.

ПРИЛОЖЕНИЕ Г

Фрагменты листинга программного кода

Проверка версии официального источника

Фрагмент реализации контроля версии источника приведен в листинге 4.1.
Листинг 4.1 – Реализация контроля версии источника

```
@staticmethod
def _enforce_version_guard(
    *, rows: list[RetrievedChunk], pg_version: str
) -> list[RetrievedChunk]:
    expected_token = f"/docs/{pg_version}/"
    filtered: list[RetrievedChunk] = []

    for item in rows:
        if (
            item.corpus_type == CorpusType.OFFICIAL.value
            and expected_token not in item.source_url
        ):
            continue
        filtered.append(item)
    return filtered
```

Выбор корпусов по режиму ответа

Фрагмент правила выбора корпусов по режиму ответа приведен в листинге 4.2.

Листинг 4.2 – Правило выбора корпусов

```
1 @staticmethod
2 def resolve_corpora(answer_mode: str) -> list[str]:
```

```
3  if answer_mode == ModeType.TUTORIAL.value:
4      return [CorpusType.OFFICIAL.value, CorpusType.SUPPLEMENTARY.value]
5  return [CorpusType.OFFICIAL.value]
```

ПРИЛОЖЕНИЕ Д

Дополнительные результаты тестирования

Команда запуска тестов

```
1 PYTHONDONTWRITEBYTECODE=1 .venv/bin/python -m pytest -q -p no:cacheprovider
```

Результат запуска

```
1 SSS..... [ 76%]  
2 ..... [100%]  
3 91 passed, 3 skipped in 1.00s
```